

Binary Classification Modeling
Using Logistic Regression to Build Credit Scores
Muniza Naqvi
Supervised by Jennifer Lewis Priestley, Ph.D.
Kennesaw State University
Submitted April 26, 2013
to fulfill the requirements for STAT 8330

Executive Summary

Risk assessment is crucial to a successful credit-lending business. With the immense data available to businesses today it is possible to model consumer-behavior to help determine who is more likely to default. Our response variable is a binary variable with a '0' indicating a credit-worthy customer and a '1' indicating bad-risk. Models with a binary response variable cannot be fitted a linear regression. This project covers the steps taken to successfully build a universal predictive model for credit-lending businesses to help maximize their profits. The project scope includes cleaning of variables and using imputation methods, discretizing and transforming variables, sampling datasets for training and validation purposes, the actual logistic modeling process and variable selection, and finally model evaluation. Our model promises good returns to businesses and has demonstrated it's universality by giving similar results based on two separate validation files.

Introduction

This research paper describes the process and results of developing a binary classification model, using Logistic Regression, to generate Credit Risk Scores. These scores are then used to maximize a profitability function.

The data for this project came from CompuCredit. Three datasets were provided:

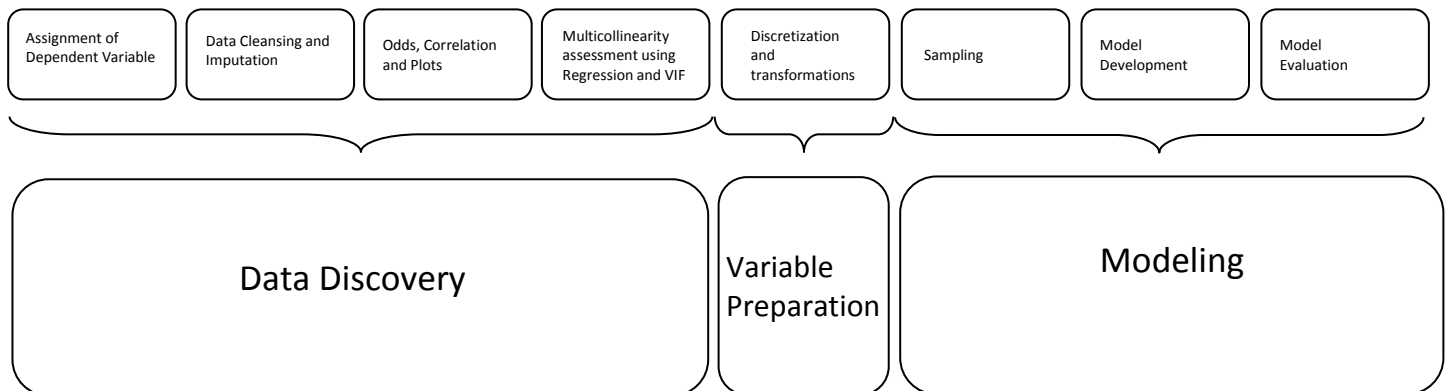
CPR. 1,462,955 observations and 338 variables. Each observation represents a unique customer. This file contains all of the potential predictors of credit performance. The variables have differing levels of completeness.

PERF. 17,244,104 observations and 18 variables. This file contains the post hoc performance data for each customer, including the response variable for modeling – DELQID.

TRAN. 8,536,608 observations and 5 variables. This file contains information on the transaction patterns of each customer.

Each file contains a consistent “matchkey” variable which was used to merge the datasets.

The process for the project included:



Each of these processes will be discussed in turn.

Data Discovery

In order to begin our modeling we had to first combine the PERF dataset, which contained all the dependent variables, with the CPR dataset which contained all predictor variables. We know that the one consistent variable between the two datasets is “matchkey”; hence it is the key to aligning transactions in both datasets without repetitions in our merged dataset. To achieve this we sorted the CPR dataset by “matchkey” to get it ready for the merging process. On the other hand we sorted the PERF dataset with respect to “matchkey” and our response variable “delqid” (delinquent ID). Since SAS overwrites the last value with previous values when merging datasets

with multiple observations for a variable this ensured that our merged dataset retained the worst score of “delqid” for each customer.

Our merged dataset had a total of 1,743,505 transactions and 356 variables. From here on we wanted to perform binary classification of our response variable “delqid”. Table 1 and Table 2 introduce our response variable “delqid”:

Table 1: Delqid Classification

Values	Definition
0	Too new to rate
1	Current
2	30 Days Late
3	60 Days Late
4	90 Days Late
5	120 Days Late
6	Charge-off, repossession, account in credit counseling, included in bankruptcy
7	No industry-specified trade, but there is a trade of any other type
8	All items that could be considered fell into an exclusion, such as disputed trades
9	No trade on file

Table 2: Frequency Table for Delqid

delqid				
DELQID	Frequency	Percent	Cumulative Frequency	Cumulative Percent
0	1124027	65.11	1124027	65.11
1	280433	16.24	1404460	81.36
2	69528	4.03	1473988	85.39
3	38530	2.23	1512518	87.62
4	29853	1.73	1542371	89.35
5	23325	1.35	1565696	90.70
6	23939	1.39	1589635	92.08
7	136640	7.92	1726275	100.00

Frequency Missing = 17230

Table 1 shows the meaning behind each digit in our response variable, and Table 2 shows the frequency counts of each class within “delqid” and displays the number of missing “delqid” observations as well (this frequency table is from before the missing values were deleted). Since “delqid” is our target variable we had no option but to delete the missing-value transactions and

as a result we were now left with 1726275 transactions. We created a new variable called “goodbad” which had two levels within it; generally good customers are those who stay within 0-2 range of the “delqid” variable, whereas risky customers range from 3-7 of the “delqid” variable. So all “delqid”s from 0-2 were classified as a ‘0’ in our new binary variable “goodbad” and remaining higher ones were classified ‘1’. Table 3 shows that 85.39% of our customers fell into the good (0) region of our “goodbad” variable. We can also see how 14.61% customers are classified bad (1); they are the ones who are likely to default on their accounts.

Table 3: Frequency Table for Goodbad

goodbad	Frequency	Percent	Cumulative Frequency	Cumulative Percent
0	1473988	85.39	1473988	85.39
1	252287	14.61	1726275	100.00

The mean for our “goodbad” variable is 0.1757, and due to the binary nature of “goodbad” this suggests that 17.57% of our customers are likely to default.

Our next challenge was to ensure our CPR dataset variables had “clean” data that made sense in context. We were informed that most variables in the CPR dataset contain default values, which means they imply something other than what their numeric value suggests; Table 4 shows a random sample of our merged data with lots of default values.

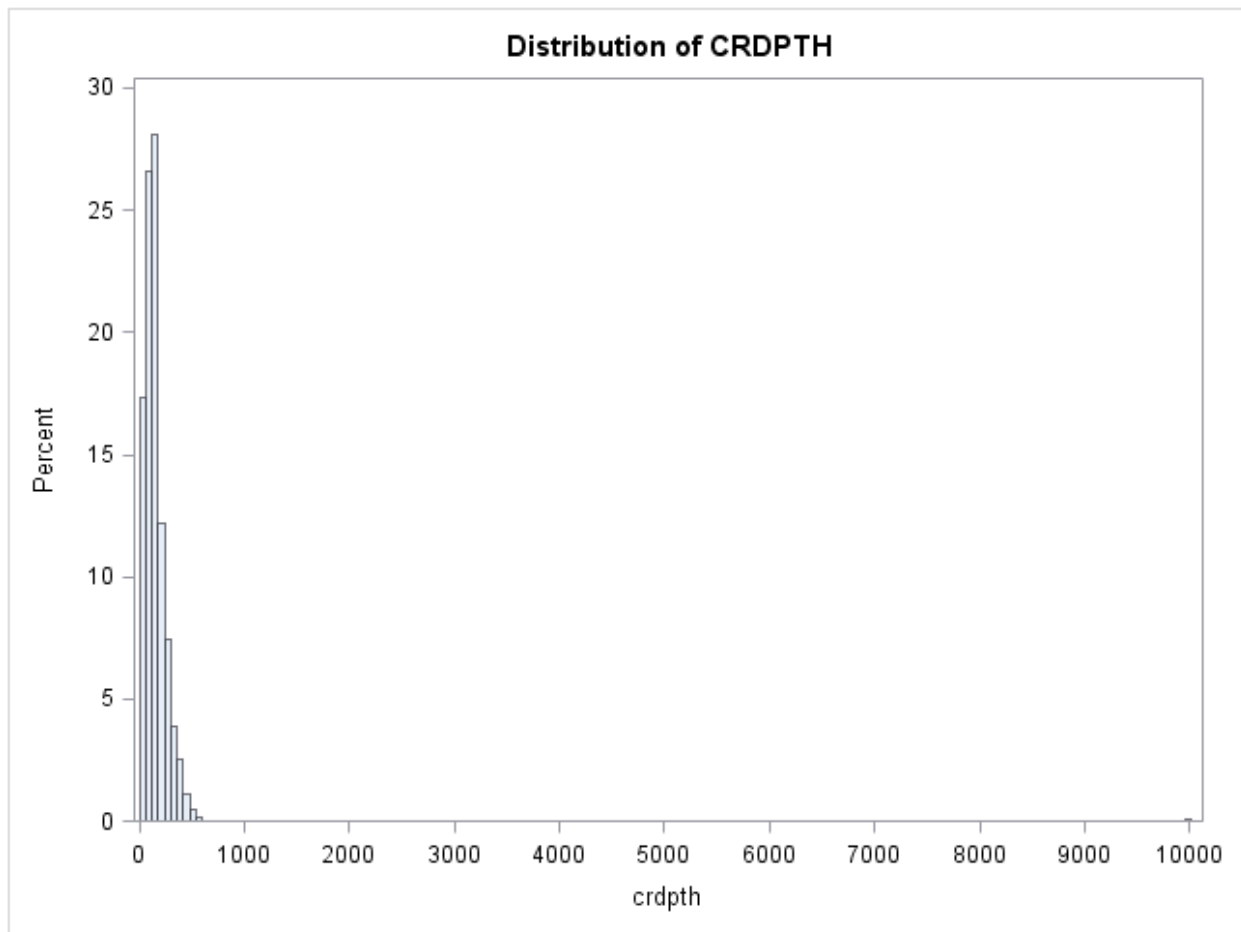
Table 4: Default Values

PFMAXH	BRMINH	DCMINH	FFMINH	BIMINH	ORMINH	ATMINH	AFMINH	CUMINH	PFMINH	BRADBM
5024	900	200	9999997	15000	970	9999997	9999998	9999998	1440	1.0187
1000	500	500	200	9999998	420	9999997	12200	9999998	1000	1.0391
1000	350	600	800	9999998	9999997	9999997	9999997	9999997	1000	0.9930
2850	250	200	2425	9999998	1140	9999997	35187	9999998	840	1.5120
4000	1854	1000	3800	9999998	9999997	9999997	9999998	9999998	1076	1.0000
9999997	350	9999997	9999997	9999997	400	9999997	9999997	9999997	9999997	1.1629
15000	1593	9999998	9999997	9999997	9999997	9999997	12868	9999997	15000	0.8569
9999996	1500	2000	9999998	9999997	9999997	9999997	9999998	9999998	9999996	0.0434
9999997	2000	400	9999998	9999998	9999998	9999997	9999997	10906	9999997	0.8810
2180	300	9999998	9999997	9999998	3000	9999997	7250	6200	2180	0.4800

For a good predictive model we need as many observations as possible, but on the other hand if the data in the observations is senseless it would lead to a useless model. So we needed to salvage our model by imputing these default values. Basic diagnostics, such as histograms, revealed that our data was skewed, thus the best option was to perform a median-based imputation. First, even though we won’t be using the variable “age” as a predictor, we needed to clean out its missing values (a total of 470846) because we noticed how if age was missing then all other variables were missing or had zero values. In this case the only option was to delete all those missing values as, and as a result we were left with 1,255,429 observations.

From here on we had a step-by-step strategy to identify and impute the median into the default values. First we ran did a univariate analysis of the individual variable, for example the histogram for “crdpth” (which is the age of a customer’s oldest trade) is displayed in Figure 1.

Fig 1: Crdpth Histogram with Defaults



Observe the extreme data-points on the extreme right; these can be better viewed in Table 5 which gives frequency counts for the last few values of “crdpth”. Note the jump from a value of 929 to 9996 – clearly the last three values are coded.

Table 5: Frequency Table for Crdpth (Partial)

815	1	0.0	99.9
851	1	0.0	99.9
893	1	0.0	99.9
905	1	0.0	99.9
912	1	0.0	99.9
929	1	0.0	99.9
9996	97	0.0	99.9
9998	27	0.0	100.0
9999	605	0.0	100.0

Crdpth N Cumulative Percent

We ran univariate analysis again, but this time only on values less than 9996, and found the actual median for “crdpth” which is 130. Figure 2 shows the histogram for “crdpth” without the coded values. We replaced all default values with the median value using a simple if-then data step statement in SAS. Table 6 shows the summary of “crdpth” before and after the imputation process. “Crdpth” will serve us as a more meaningful variable from now on.

Fig 2: Histogram for Crdpth without Defaults

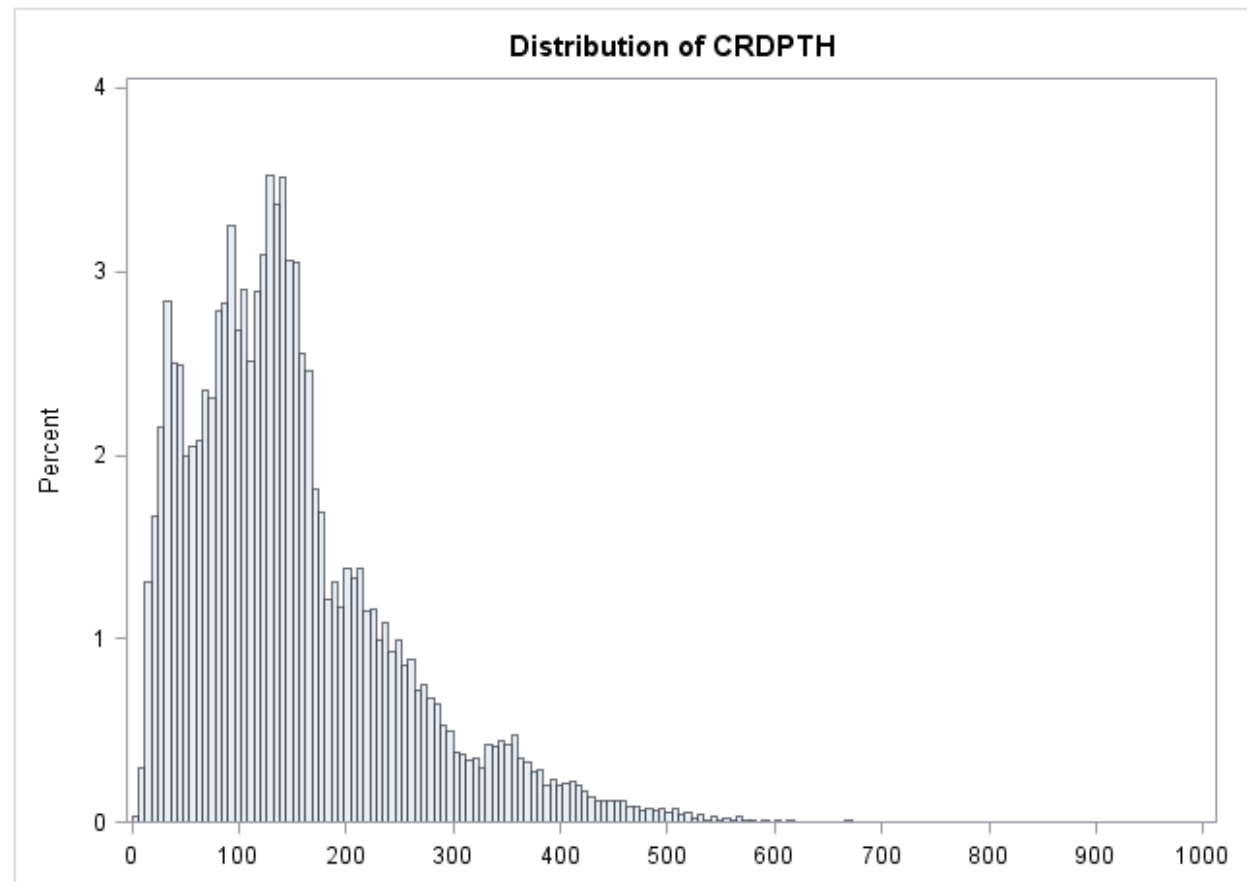


Table 6: Before & After - Crdpth

Analysis Variable : CRDPTH crdpth					
N	N Miss	Mean	Median	Minimum	Maximum
1255429	470846	154.0329704	131.0000000	0	9999.00

Analysis Variable : CRDPTH crdpth					
N	N Miss	Mean	Median	Minimum	Maximum
1255429	470846	148.3025125	130.0000000	0	929.0000000

Another variable that was imputed was “inqage”, which denotes age of most recent inquiry. Figure 3 shows the spread of “inqage” with defaults; note the extreme values on the right – these are the default/coded values. Figure 4 shows the histogram without the default values. The extreme defaults made the variable look almost binary in nature.

Fig 3: Histogram for Inqage with Defaults

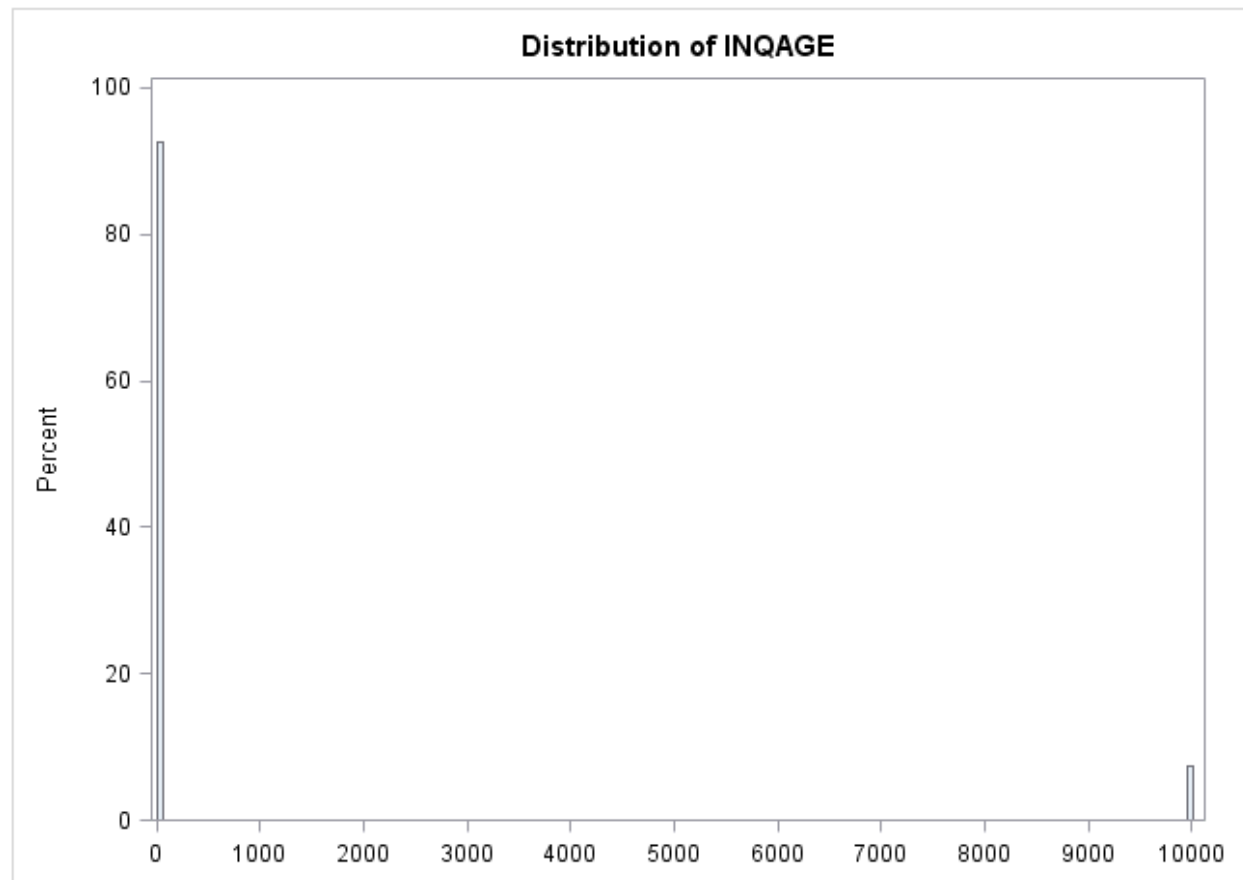
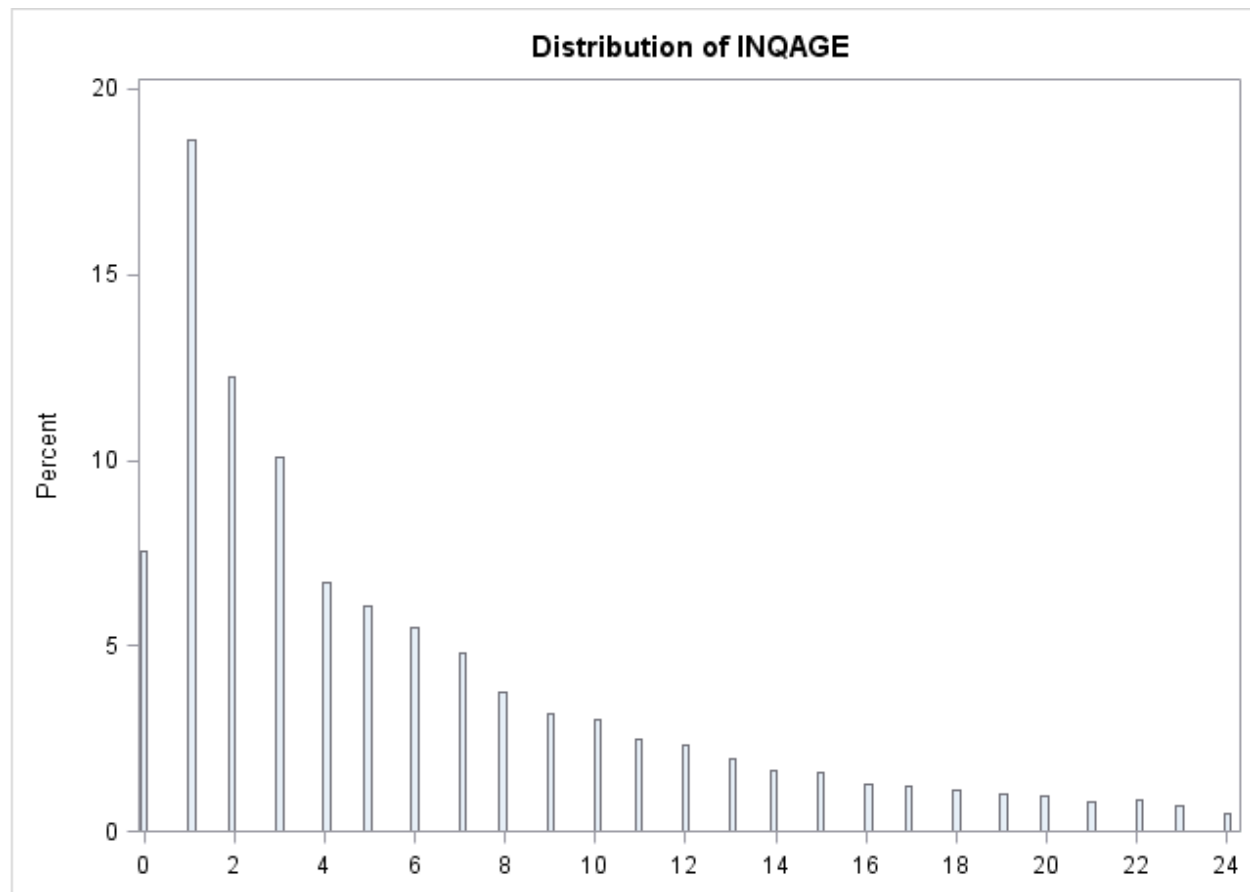


Fig 5: Histogram of Inqage without Defaults



We found the median for “inqage” to be 4 and imputed the default values with it. Table 7 shows the before and after imputation results for “inqage”. The change in its mean value shows the magnitude of the effect of imputation on “inqage”; it went from being 754.18 to 5.63.

Table 7: Before & After Imputation - Inqage

Analysis Variable : INQAGE inqage					
N	N Miss	Mean	Median	Minimum	Maximum
1255429	470846	754.1820812	4.0000000	0	9999.00

Analysis Variable : INQAGE inqage					
N	N Miss	Mean	Median	Minimum	Maximum
1255429	470846	5.6332664	4.0000000	0	24.0000000

To conclude the importance of the imputation process, see the histograms (Figure 5 and 6) for “dcpdbal” (department store accounts past due). The median went from being 93 to 1 after imputation (see Table 8).

Fig 6: Histogram for Dcpdbal with Defaults

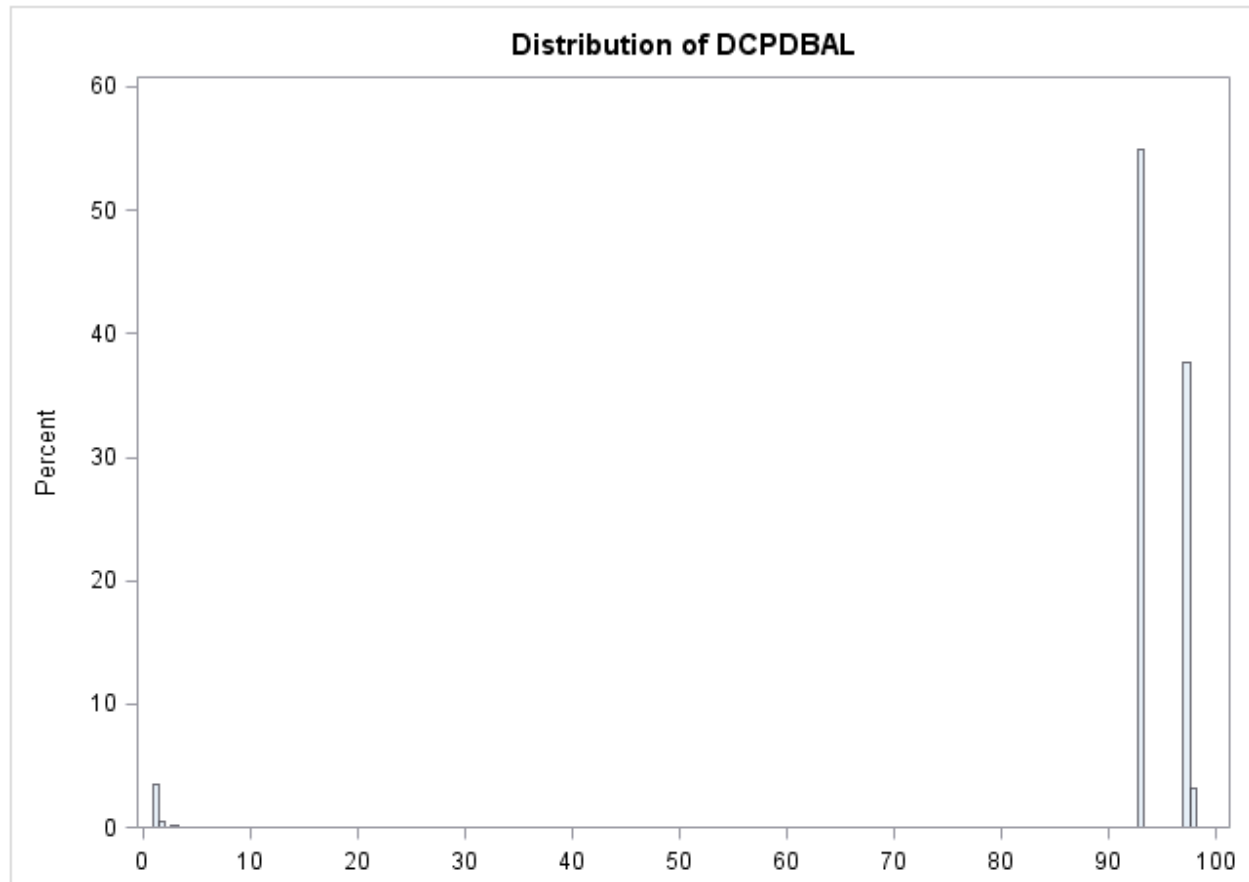


Fig 7: Histogram for Dcpdbal without Defaults

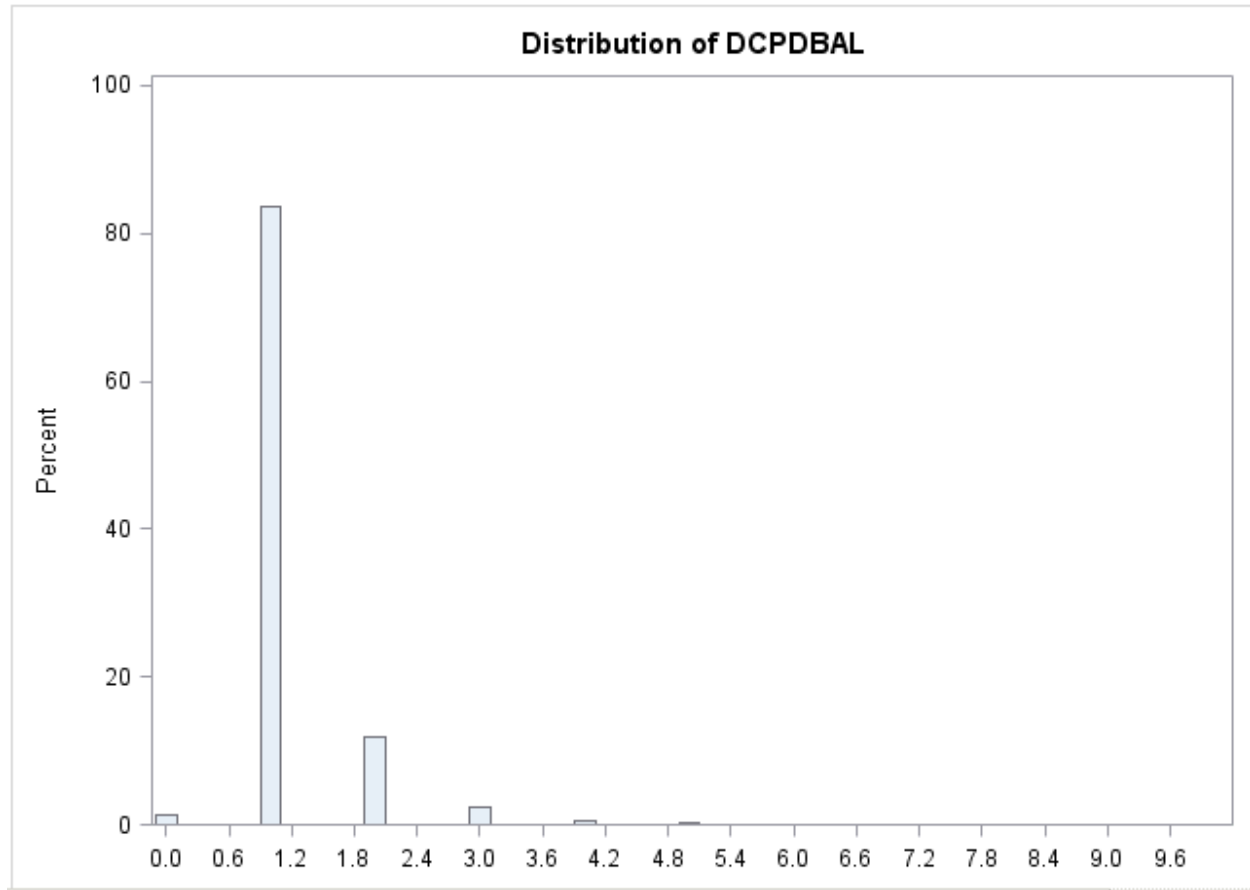


Table 8: Before & After - Dcpdbal

Analysis Variable : DCPDBAL dcpdbal					
N	N Miss	Mean	Median	Minimum	Maximum
1255429	470846	90.8310872	93.0000000	0	99.0000000

Analysis Variable : DCPDBAL dcpdbal					
N	N Miss	Mean	Median	Minimum	Maximum
1255429	470846	1.0076189	1.0000000	0	10.0000000

We repeated this process for a total of 20 variables, shown in Table 9. However, when dealing with more than 300 variables and time limitations it is best to automate the process. Hence we used a pre-programmed macro to impute the remaining variables.

Table 9: Before & After - Manually Imputed Variables

Variable	Label	N	N Miss	Mean	Median	Minimum	Maximum
TRADES	trades	1255429	470846	18.7817415	17.0000000	1.0000000	99.0000000
RBAL	rbal	1255429	470846	122189.84	4592.00	0	9999999.00
DCBAL	dcbal	1255429	470846	4674151.89	2503.00	0	9999999.00
AFADBM	afadbm	1255429	470846	7.7828010	9.9997000	0	9.9999000
AMTPD	amtpd	1255429	470846	6264956.52	9999993.00	0	9999999.00
ATPDBAL	atpdbal	1255429	470846	96.4883598	97.0000000	0	99.0000000
AVGMOS	avgmos	1255429	470846	59.2374806	56.0000000	0	999.0000000
BRAGE	brage	1255429	470846	127.0679306	91.0000000	0	9999.00
COLLS	colls	1255429	470846	0.7430185	0	0	99.0000000
CRDPATH	crdpth	1255429	470846	154.0329704	131.0000000	0	9999.00
CUOPEN	cuopen	1255429	470846	52.3743254	97.0000000	0	99.0000000
DCAGE	dcage	1255429	470846	4018.29	179.0000000	0	9999.00
DCHIC	dchic	1255429	470846	4690980.92	8300.00	0	9999999.00
DCPDBAL	dcpdbal	1255429	470846	90.8310872	93.0000000	0	99.0000000
FFADB	ffadb	1255429	470846	4.7659044	0.9119000	0	9.9999000
FFAGE	ffage	1255429	470846	4069.31	148.0000000	0	9999.00
INQAGE	inqage	1255429	470846	754.1820812	4.0000000	0	9999.00
PDBAL	pdbal	1255429	470846	59.1878808	93.0000000	0	99.0000000
TADB	tadb	1255429	470846	0.5649129	0.5766000	0	9.9999000
TPOPEN	tpopen	1255429	470846	0.5805639	0.5625000	0	9.9999000

Variable	Label	N	N Miss	Mean	Median	Minimum	Maximum
TRADES	trades	1255429	0	18.7422252	17.0000000	1.0000000	92.0000000
RBAL	rbal	1255429	0	7436.22	4505.00	0	279747.00
DCBAL	dcbal	1255429	0	354.4767072	81.0000000	0	54001.00
AFADBM	afadbm	1255429	0	0.7618596	0.7795000	0	9.9992000
AMTPD	amtpd	1255429	0	1627.10	800.0000000	0	9999992.00
ATPDBAL	atpdbal	1255429	0	0.9999339	1.0000000	0	3.0000000
AVGMOS	avgmos	1255429	0	58.6901545	56.0000000	0	581.0000000
BRAGE	brage	1255429	0	105.1865307	91.0000000	0	746.0000000
COLLS	colls	1255429	0	0.7016183	0	0	98.0000000
CRDPATH	crdpth	1255429	0	148.3025125	130.0000000	0	929.0000000
CUOPEN	cuopen	1255429	0	1.0000669	1.0000000	0	39.0000000
DCAGE	dcage	1255429	0	99.2999493	84.0000000	0	791.0000000
DCHIC	dchic	1255429	0	1939.77	1308.00	0	71584.00
DCPDBAL	dcpdbal	1255429	0	1.0076189	1.0000000	0	10.0000000
FFADB	ffadb	1255429	0	0.1883744	0.0867000	0	9.9992000
FFAGE	ffage	1255429	0	75.4666588	60.0000000	0	912.0000000
INQAGE	inqage	1255429	0	5.6332664	4.0000000	0	24.0000000
PDBAL	pdbal	1255429	0	2.1838288	2.0000000	0	47.0000000
TADB	tadb	1255429	0	0.5447650	0.5757000	0	9.9992000
TPOPEN	tpopen	1255429	0	0.5698415	0.5600000	0	1.0000000

The macro was fed details for all default values and programmed to find and impute the median for the default values. We not only had the defaults removed with help from the macro, but also had it delete all variables that needed imputation for more than 30% values. This number was chosen after a few different runs ranging from 60% to 10%. 30% gave us a good return of variables, which didn't need more than 30% of their observations imputed. If a variable was missing more than 30% of it's data and we still imputed it with it's median, then we would pretty much be defining our own values for that variable, and it would not be a useful real-world predictor. After the macro had been run we were left with 150 variables. Table 10 shows the macros successful imputation for the variable "cuopen", which is number of open credit union accounts. Figures 8 and 9 show it's histogram with and without defaults respectively. The maximum dropped from 99 to 39 having a drastic effect on the mean and median for the variable.

Table 10: Before & After Cuopen

Analysis Variable : CUOPEN cuopen					
N	N Miss	Mean	Median	Minimum	Maximum
1255429	470846	52.3743254	97.0000000	0	99.0000000

Analysis Variable : CUOPEN cuopen					
N	N Miss	Mean	Median	Minimum	Maximum
1255429	470846	1.0000669	1.0000000	0	39.0000000

Fig 8: Histogram of Cuopen with Defaults

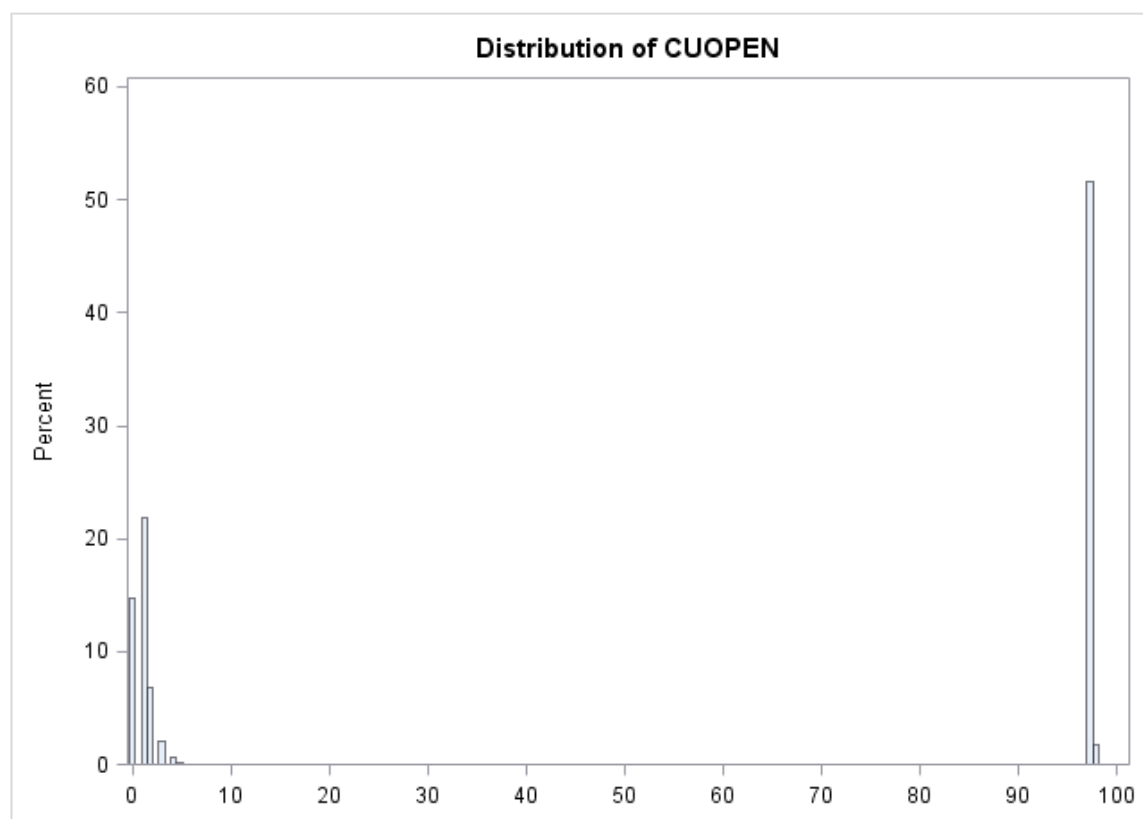
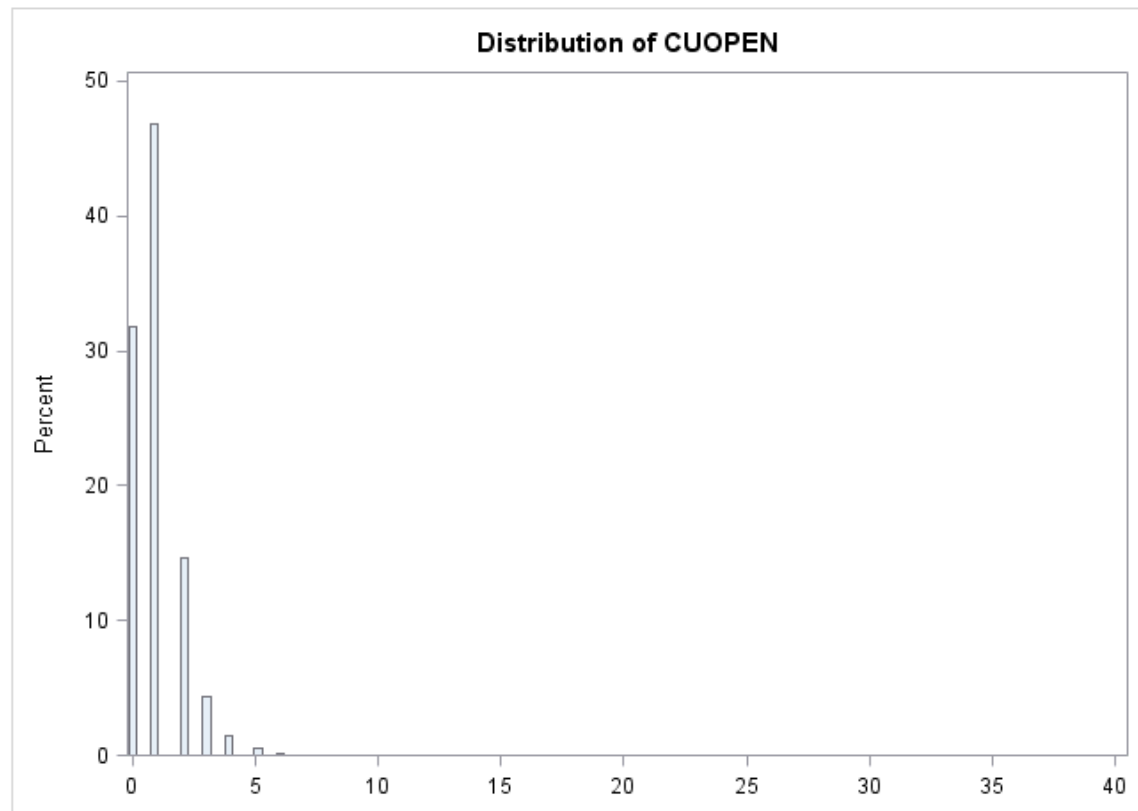


Fig 9: Histogram of Cuopen without Defaults



The appropriate final logistic model would require further transformation and discretization of the remaining variables. But looking at these variables it is apparent that there exist linear combinations and multicollinearity among them; such as “badpr2” which is the number of bad debt public records, and “nmprbd” which is the number of public records less medical bills plus bad debts; “nmprbd” is a linear combination of “badpr2” (SAS detects and reports linear combinations as well). To find the multicollinearity we ran ordinary least squares regression modeling on the remaining CPR dataset variables in our merged dataset to “delqid” and looked for and eliminated any variables that had more than 10 VIF (variance inflation factor). A VIF of 10 or more for a variable suggests that 90% (or more if $VIF > 10$) variance (or spread of observations) in the model is already being explained by other variables and that the standard error of the variables coefficient is inflated. A high VIF makes the variables t-statistic go near zero and as a result makes its p-value sky-rocket leading to statistically insignificant results. Hence it isn’t adding any useful information to our model and should be removed. VIF’s can be calculated by modeling individual variables as response and using their R^2 values in the following formula: $VIF = \frac{1}{1-R^2}$. For this project we used the SAS option VIF to find VIF’s for all remaining variables. A few very high VIF’s are displayed in Table 11 which were removed. Note the insignificant p-values for these variables.

Table 11: Very High VIF's

Variable		DF	Parameter Estimate	Standard Error	t-value	p-value	VIF
BRTRADES	btrades	1	-0.01240	0.00967	-1.28	0.1999	3575.96790
TRATE79	trate79	1	0.01077	0.00419	2.57	0.0102	303.24374

Table 12 shows the ANOVA Table for the “delqid” model with all macro-retained variables from the CPR dataset. And Table 13 shows the ANOVA Table the remaining variables after all high VIF’s were removed.

Table 12: ANOVA with High VIF's

Analysis of Variance					
Source	DF	Sum of Squares	Mean Square	F Value	Pr > F
Model	150	5748765	38325	77965.4	<.0001
Error	1.26E6	617051	0.49157		
Corrected Total	1.26E6	6365816			

Root MSE	0.70112	R-Square	0.9031
Dependent Mean	1.29532	Adj R-Sq	0.9031
Coeff Var	54.12685		

Table 13: ANOVA for less than 10 VIF's

Analysis of Variance					
Source	DF	Sum of Squares	Mean Square	F Value	Pr > F
Model	99	2178769	22008	6598.20	<.0001
Error	1.26E6	4187046	3.33542		
Corrected Total	1.26E6	6365816			

Root MSE	1.82631	R-Square	0.3423
Dependent Mean	1.29532	Adj R-Sq	0.3422
Coeff Var	140.99292		

By the end of our first phase in the project, data discovery, we were left with 99 variables and 1,255,429 observations.

Edit: The regression method used had returned 99 variables. In this step we only needed 50+ variables, so we tried another approach to get rid of redundant and insignificant variables; variable clustering. Cluster analysis groups together similar variables, and shows the most significant variable, which should be retained, from each cluster. We used the option to keep a maximum of 60 clusters, since we really needed just 50+ variables. We were returned 63 significant variables, which will be used hereon. By the end of the process we had a total of 81 variables, where 18 were from the perf data set. The table below shows the clustering process. Cluster 4 identifies all the variables listed under it as collinear and hence chooses the best which is “Crate79”. This process chooses the variable with least variance, or the with the highest associated eigen-value.

Cluster	Variable	R-square Ratio
---------	----------	----------------

Cluster 4	CRATE79	0.1719
	TRATE79	0.1732
	TRR39	0.2151
	TRR49	0.0936
	TRR29	0.5963
	TRCR39	0.1107
	TRCR49	0.0961
Cluster 5	CRATE1	0.1151
	TRATE1	0.1140
	TR124	0.3719
	TROPENEX	1.0126
Cluster 6	CRDPTH	0.2242
	AVGMOS	0.2092

Variable Preparation

After the cleaning and weeding out of redundant variables the next required step is to discretize the remaining variables. Most of the variables we have are continuous whereas some are not. Since our data is not normally distributed or linearly related, and doesn't have equal variance in each group we will be utilizing logistic regression. To capture the best mathematical relationship we will need to first turn our continuous variables into discrete ordinal variables and then figure out the most appropriate transformation.

Two different discretization approaches were utilized for this purpose. The first involved manual analysis of each continuous variable, and subsequent user-defined categorization of the variable into ordinal classes with equal widths. The second one was more automated, where the analyst set the number of groups in for each variable and SAS assigned the partitions among the groups such that the groups will have equal frequencies.

In both steps we also calculate the odds of each variable since that provides a more meaningful explanation of in-between values in discrete variables. The odds for each variable were calculated against the "Goodbad" variable; hence this would give us insight into what kind of relationship the given variable had with the chances of defaulting. The log of odds was also calculated as this transformation helped normalizing our variable.

Discretization 1 – The Process

To get an idea of how a variable was behaving we first acquired a histogram for it. Figure 1 shows the histogram for variable "BRAGE" which shows the age of oldest revolving bank account. The distribution of "BRAGE" is right-skewed and multi-modal, and you can see how "BRAGE" is a continuous variable. Along with the histogram, a quick look at the minimum and maximum for each variable gave us an idea of how to categorize the variable into equal widths. In the case of "ordBRAGE" – the ordinal version – was categorized into 11 groups each with a width of 50 ranging from 0 to 746 as shown in table 1. Since the widths were set by the analyst, equal frequency wasn't guaranteed as shown in table 1; Group1 has 324154 observations, whereas group 11 has only 1057. Figure 2 shows the histogram for the ordinal version of "Brage"; it is still right-skewed but unimodal. Next we ran quantitative analysis on "ordBRAGE" with respect to our "Goodbad" variable to obtain the mean "Goodbad" at each level of the newly created ordinal. Table 2 shows the average "Goodbad" values at each levels of "ordBrage"- notice the decrease in mean "Goodbad" as "ordBrage" increases. This relationship is plotted in figure 3: plot of mean "Goodbad" versus "ordBrage". From the plot we can see that the relationship between mean "Goodbad" and "ordBrage" is strong for the most part suggesting that as the age of oldest revolving bank account goes up, the chances of default go down. However the relationship is not entirely linear, which is a problem that may go away with a log transformation.

Figure 1: BRAGE Histogram

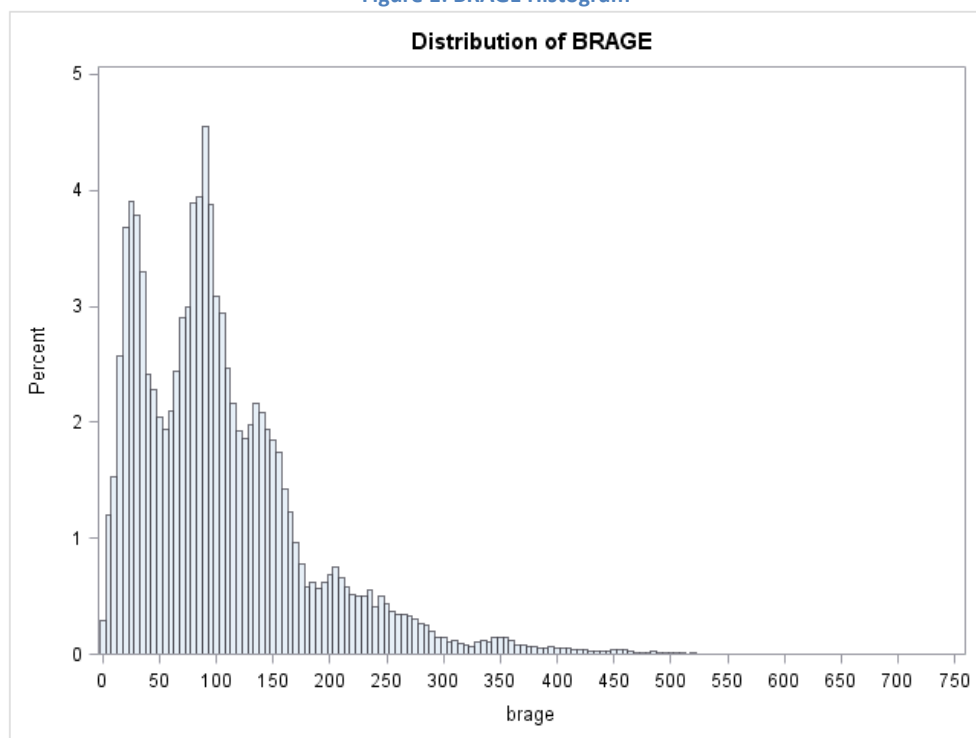


Table 1: Descriptive Statistics of ordBRAGE

The SAS System

The MEANS Procedure

Analysis Variable : BRAGE brage						
ordBRAGE	N Obs	N	Mean	Std Dev	Minimum	Maximum
1	324154	324154	27.2421781	11.7390594	0	49.0000000
2	391998	391998	78.2501365	13.6513264	50.0000000	99.0000000
3	274410	274410	122.6570424	14.7503603	100.0000000	149.0000000
4	125451	125451	168.3381878	14.3012301	150.0000000	199.0000000
5	70205	70205	222.4563920	14.6356719	200.0000000	249.0000000
6	35873	35873	270.6720096	13.6347359	250.0000000	299.0000000
7	13565	13565	325.4861039	15.3798859	300.0000000	349.0000000
8	10866	10866	369.7101049	14.7175175	350.0000000	399.0000000
9	5413	5413	423.7613154	16.5311282	400.0000000	459.0000000
10	2437	2437	472.4189577	13.0603507	450.0000000	499.0000000
11	1057	1057	537.5477767	36.7518016	500.0000000	746.0000000

Figure 2: Histogram of ordBrage

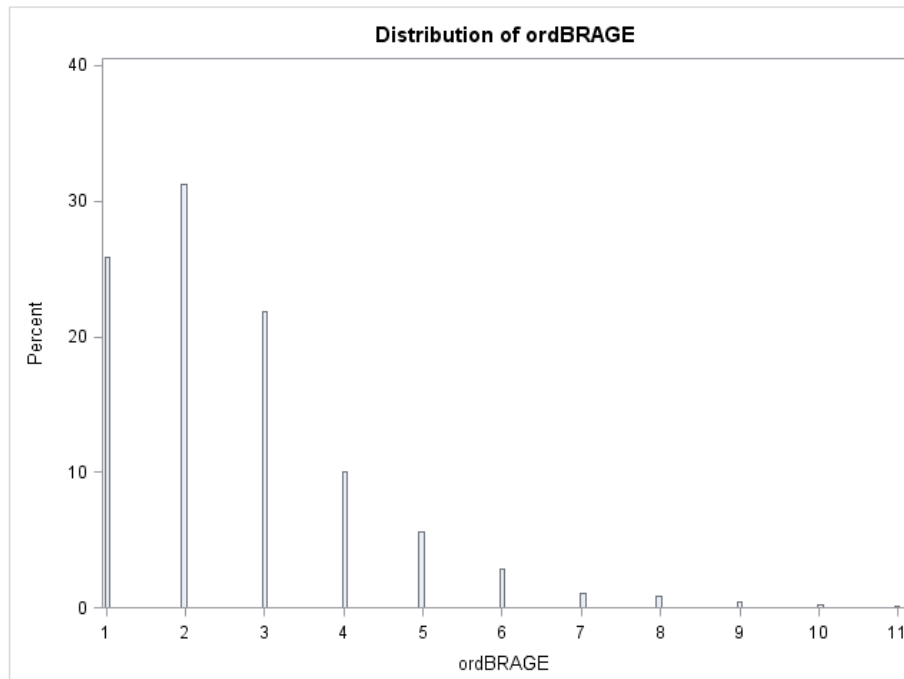
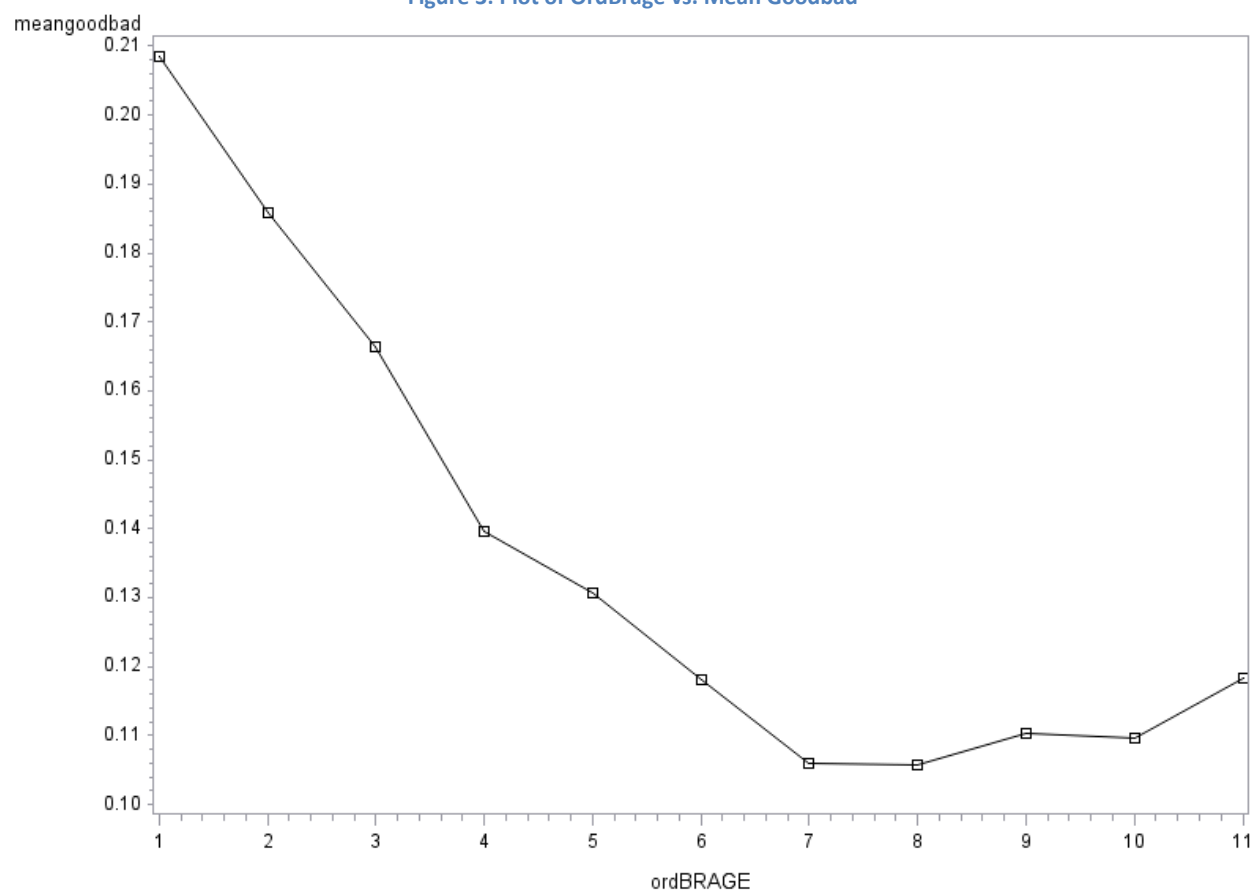


Table 2: OrdBrage and Mean of Goodbad

Analysis Variable : goodbad						
ordBRAGE	N Obs	N	Mean	Std Dev	Minimum	Maximum
1	324154	324154	0.2084873	0.4062276	0	1.0000000
2	391998	391998	0.1859168	0.3890400	0	1.0000000
3	274410	274410	0.1662840	0.3723361	0	1.0000000
4	125451	125451	0.1397039	0.3466810	0	1.0000000
5	70205	70205	0.1306032	0.3369683	0	1.0000000
6	35873	35873	0.1180832	0.3227112	0	1.0000000
7	13565	13565	0.1060818	0.3079537	0	1.0000000
8	10866	10866	0.1058347	0.3076401	0	1.0000000
9	5413	5413	0.1102900	0.3132799	0	1.0000000
10	2437	2437	0.1095609	0.3124058	0	1.0000000
11	1057	1057	0.1182592	0.3230677	0	1.0000000

Figure 3: Plot of OrdBrage vs. Mean Goodbad



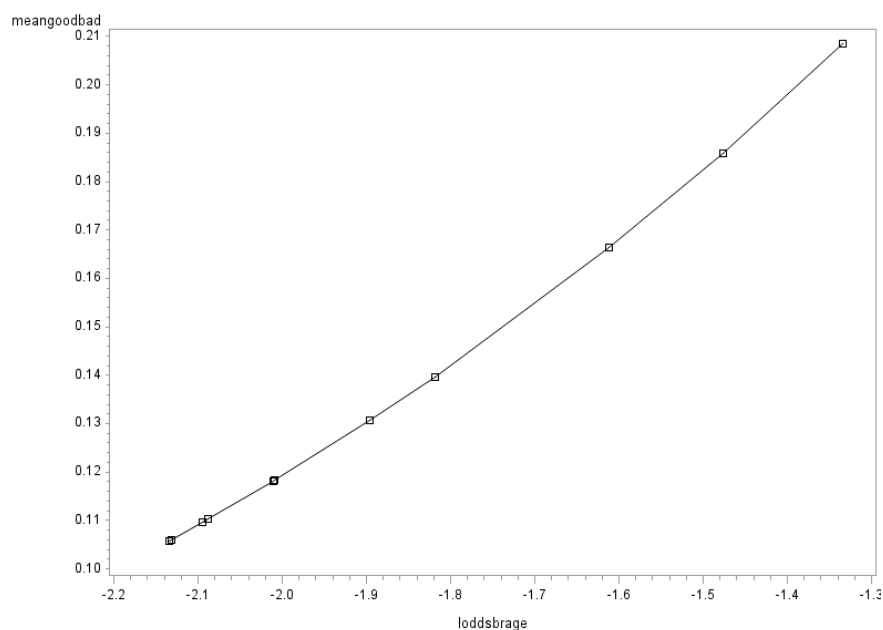
The next phase of discretization 1 involved calculating odds and the log of odds for the ordinal versions of our variables. The odds of an event can be mathematically expressed as $Odds = \frac{p}{1-p}$ where p is the probability in favor of the event. In context, this gives us the odds of an individual defaulting. The natural log of the odds gives us a normalized version of the odds variable. Table 3 shows the odds, log of odds and a test variable which shows the inverse of odds (odds of not defaulting) for “ordBrage”. From this table we can see that individuals who fell in the 3rd class of “ordBrage” are 0.20:1 times likely to default and the odds of them not defaulting are 5:1.

Table 3: Summary of Odds and Log of Odds for ordBrage

Obs	ordBRAGE	_TYPE_	_FREQ_	avg_ind	avg_dep	oddsbrage	loddsbrage	testbrage
1	.	0	1255429	105	0.17572	0.21318	-1.54564	4.69097
2	1	1	324154	27	0.20849	0.26340	-1.33407	3.79645
3	2	1	391998	78	0.18592	0.22838	-1.47676	4.37875
4	3	1	274410	123	0.16628	0.19945	-1.61220	5.01381
5	4	1	125451	168	0.13970	0.16239	-1.81775	6.15799
6	5	1	70205	222	0.13060	0.15022	-1.89564	6.65678
7	6	1	35873	271	0.11808	0.13389	-2.01071	7.46860
8	7	1	13565	325	0.10608	0.11867	-2.13140	8.42669
9	8	1	10866	370	0.10583	0.11836	-2.13401	8.44870
10	9	1	5413	424	0.11029	0.12396	-2.08778	8.06700
11	10	1	2437	472	0.10956	0.12304	-2.09523	8.12734
12	11	1	1057	538	0.11826	0.13412	-2.00902	7.45600

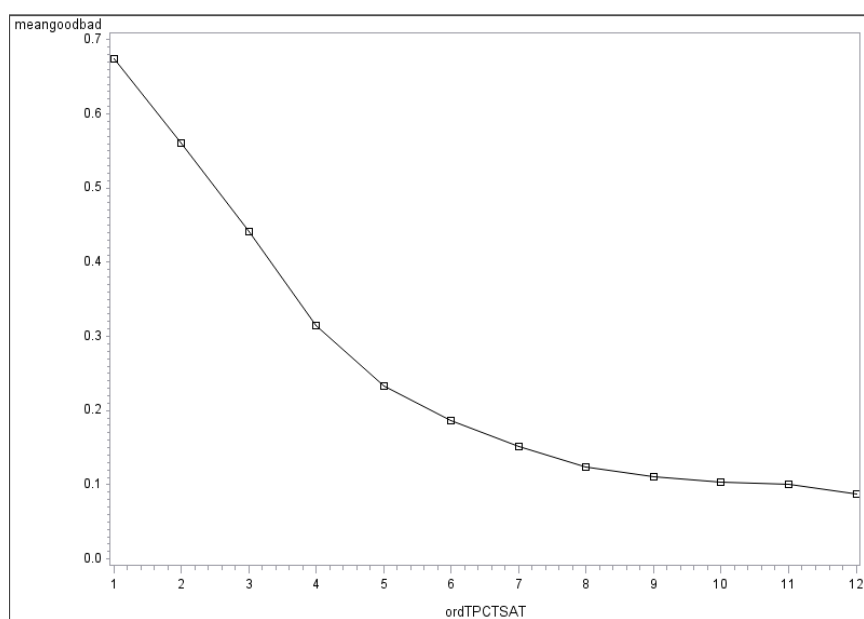
Figure 4 shows the plot of “oddsBrage” vs. mean “Goodbad”. It is evident how the log transformation has helped linearize that relationship.

Figure 4: LoddsBrage vs. mean Goodbad



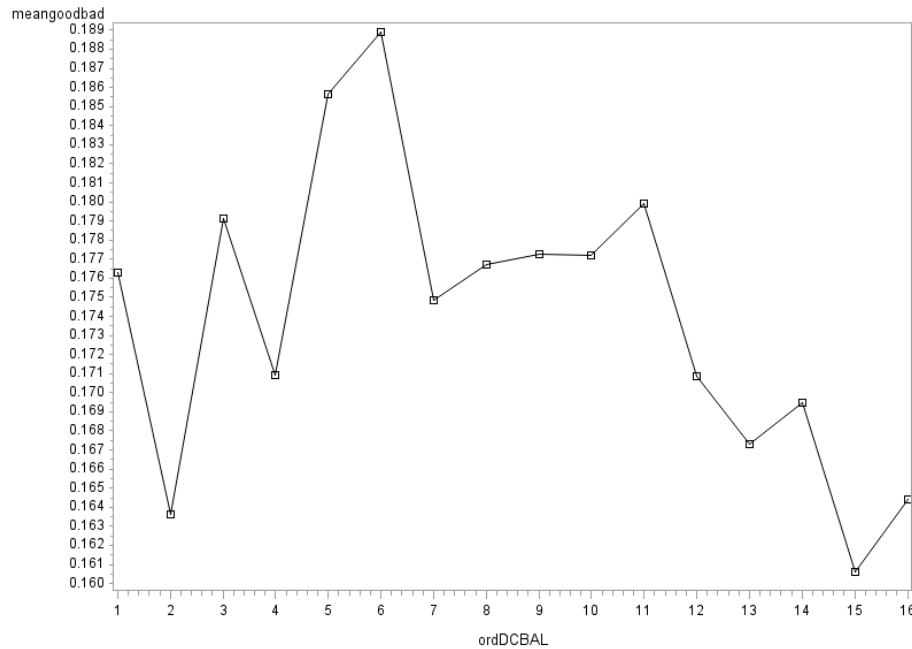
One of the good potential predictor we came across during this process was “ordTpctsat”; which is the percent of “SATS/TO” total trade. Figure 5 shows the plot of “ordTpctsat” versus mean “Goodbad”. Although not entirely linear, it has a good one-way direction suggesting a strong relationship. An individual in group 1 of “Tpctsat” would be predicted as being a “bad risk”, whereas individuals in group 12 are potentially “good risks”. The range in y (0.1, 0.7) that this variable has is also a good indication of it being a useful predictor.

Figure 5: ordTpctsat vs. mean Goodbad



On the other hand, “ordDcbal”, in figure 6, is bouncing erratically between each level which suggests a lack of a strong predictive relationship.

Figure 6: ordDcbal vs. mean Goodbad



This process was repeated for all our variables except the ones which were already in their binary or ordinal form. Some ordinal variables, such as “Crate1” needed reduction in their number of categories to become better predictors.

Figure 7: Crate1 - Original Distribution

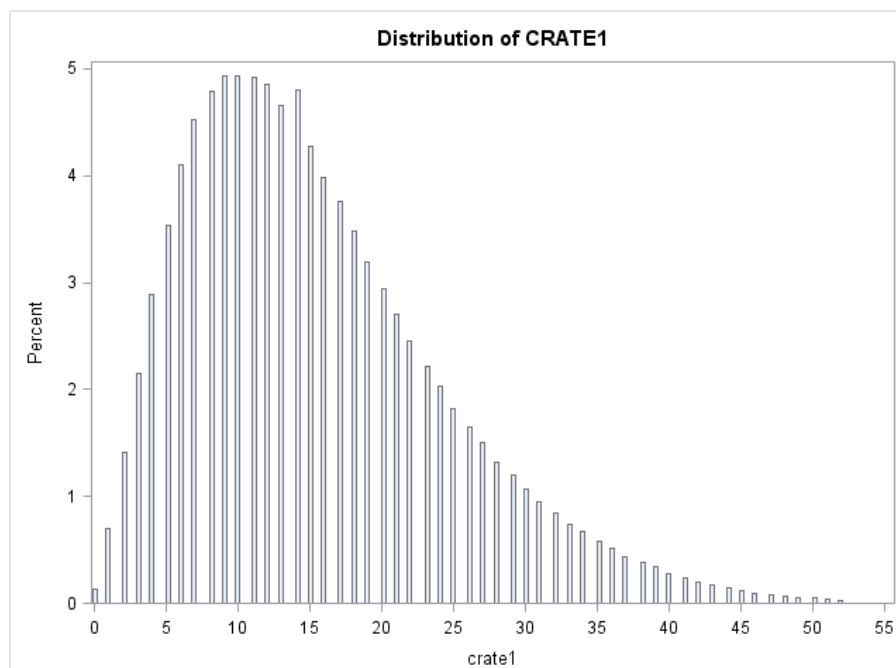
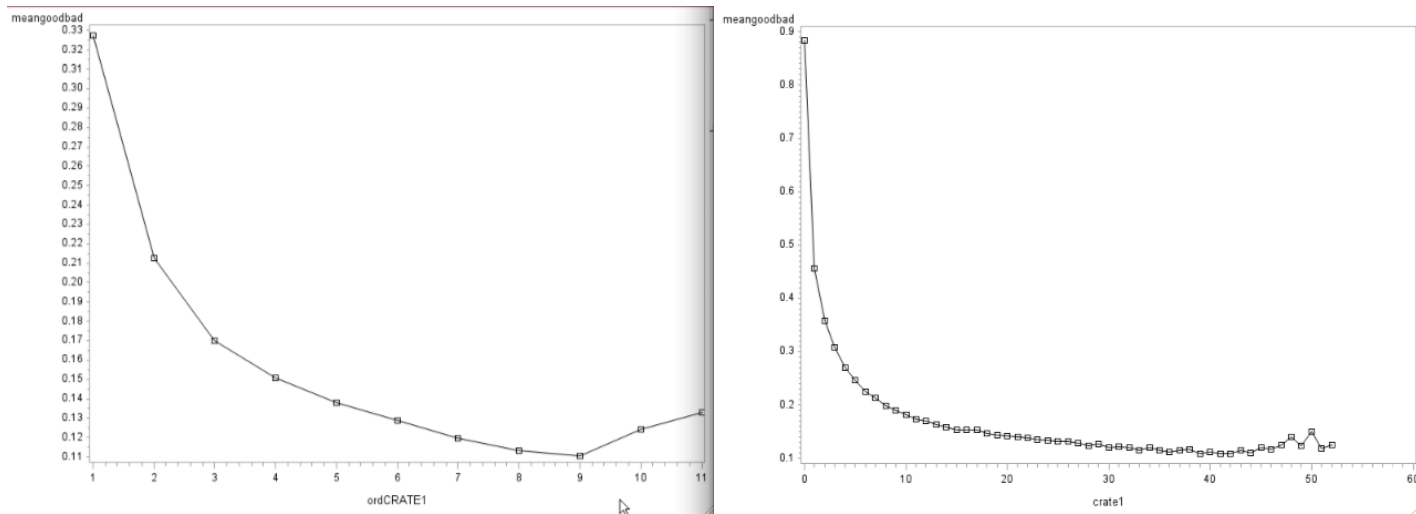


Figure 8: ordCrate1 vs. Crate1



Discretization 2 – The Process

This method utilized SAS’s built-in Rank command which partitions observations into groups based on ranks. We used the option groups to create deciles for our variables. Table 4 shows the ranking for “Brcrate1”; SAS has tried to maintain equal frequencies across the ten groups, so that each group was equally represented.

Table 4: Ranks for Brcrate1

Rank for Variable BRCRATE1				
rank	Frequency	Percent	Cumulative Frequency	Cumulative Percent
0	78617	6.26	78617	6.26
1	95082	7.57	173699	13.84
2	121721	9.70	295420	23.53
3	135749	10.81	431169	34.34
4	135823	10.82	566992	45.16
5	135366	10.78	702358	55.95
6	110462	8.80	812820	64.74
7	169598	13.51	982418	78.25
8	111196	8.86	1093614	87.11
9	161815	12.89	1255429	100.00

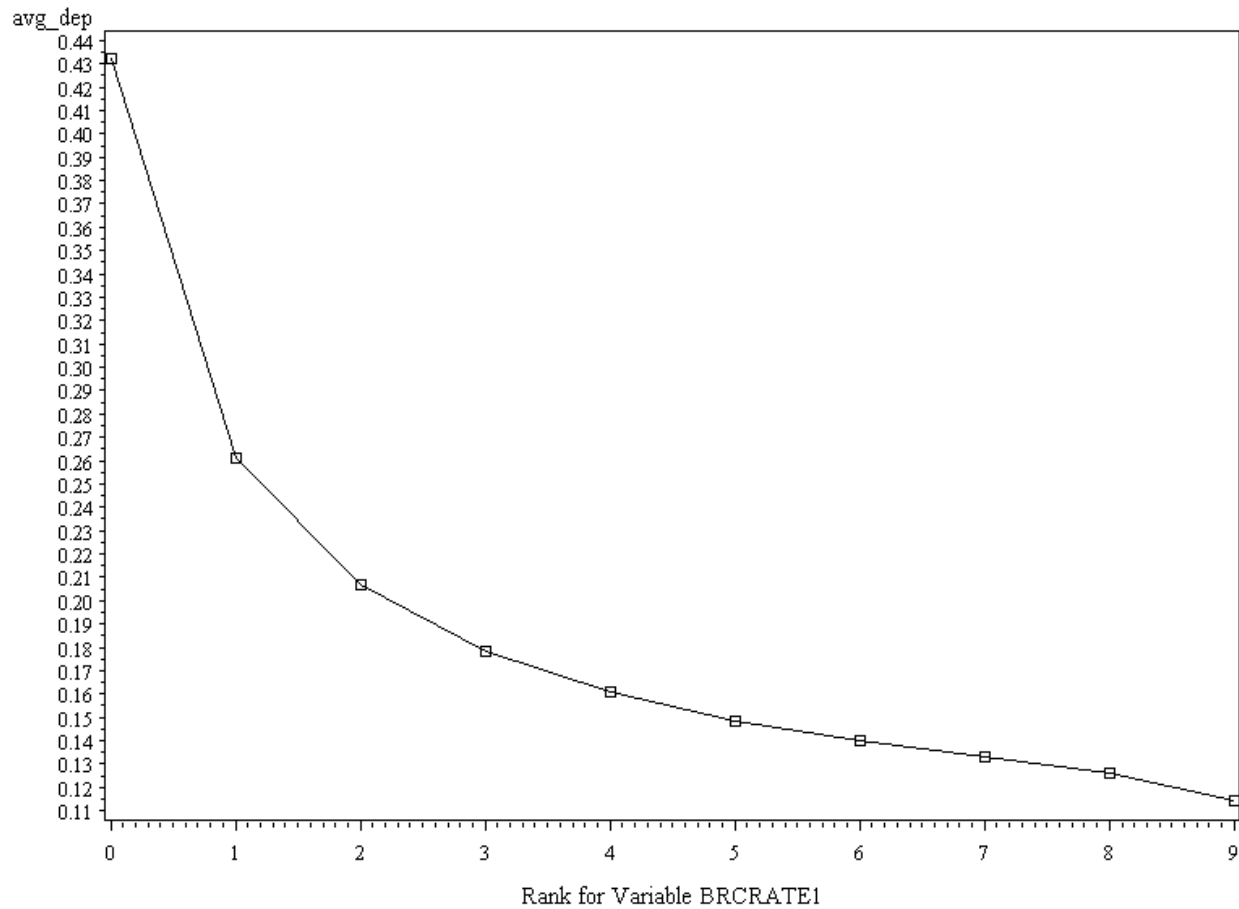
Next, a macro was ran on this variable to perform t-tests and determine significant groupings. The insignificant ones were dropped (see “Tsbal” example in table 7 and 8). In the case of “Brcrate1” all ten groups were found significant and hence retained. Table 5 shows the mean “Goodbad” for each of the groupings of – we see a decrease in mean-Goodbad as the ranks increase for “Brcrate1”. This relationship can be seen in figure 9, which shows a cascading

relationship between the two, and also sweeps up a good range for the “Goodbad” variable. This indicates that “Brcrate1” might make a good predictor in our final model.

Table 5: Brcrate1 and Mean Goodbad

Analysis Variable : goodbad						
Rank for Variable BRCRATE1	N Obs	N	Mean	Std Dev	Minimum	Maximum
0	78617	78617	0.4323111	0.4954002	0	1.0000000
1	95082	95082	0.2609116	0.4391341	0	1.0000000
2	121721	121721	0.2067269	0.4049595	0	1.0000000
3	135749	135749	0.1783733	0.3828281	0	1.0000000
4	135823	135823	0.1607533	0.3673046	0	1.0000000
5	135366	135366	0.1481761	0.3552757	0	1.0000000
6	110462	110462	0.1398128	0.3467943	0	1.0000000
7	169598	169598	0.1332740	0.3398716	0	1.0000000
8	111196	111196	0.1262905	0.3321780	0	1.0000000
9	161815	161815	0.1139944	0.3178054	0	1.0000000

Figure 9: Bcrate1 vs. Mean Goodbad



From hereon, the significant groups were turned into ordinals for the variables (numbered 1 onwards), and for the same purposes as in discretization 1 the odds and log of odds were calculated for the ordinals (see Table 6).

Table 6: OrdBrcrate1, OddsBrcrate1 & LogBrcrate1

ordeqbrcrate1	Frequency	Percent	Cumulative Frequency	Cumulative Percent
1	78617	6.26	78617	6.26
2	95082	7.57	173699	13.84
3	121721	9.70	295420	23.53
4	135749	10.81	431169	34.34
5	135823	10.82	566992	45.16
6	135366	10.78	702358	55.95
7	110462	8.80	812820	64.74
8	169598	13.51	982418	78.25
9	111196	8.86	1093614	87.11
10	161815	12.89	1255429	100.00

odseqbrcrate1	Frequency	Percent	Cumulative Frequency	Cumulative Percent
0.1286610076	161815	12.89	161815	12.89
0.1445452019	111196	8.86	273011	21.75
0.1537671349	169598	13.51	442609	35.26
0.1625376245	110462	8.80	553071	44.05
0.1739515038	135366	10.78	688437	54.84
0.1915447982	135823	10.82	824260	65.66
0.217097772	135749	10.81	960009	76.47
0.2605998467	121721	9.70	1081730	86.16
0.353018186	95082	7.57	1176812	93.74
0.7615281201	78617	6.26	1255429	100.00

lodseqbrcrate1	Frequency	Percent	Cumulative Frequency	Cumulative Percent
-2.050574181	161815	12.89	161815	12.89
-1.934163004	111196	8.86	273011	21.75
-1.872315932	169598	13.51	442609	35.26
-1.816845769	110462	8.80	553071	44.05
-1.748978732	135366	10.78	688437	54.84
-1.652633565	135823	10.82	824260	65.66
-1.527407465	135749	10.81	960009	76.47
-1.344769202	121721	9.70	1081730	86.16
-1.041235705	95082	7.57	1176812	93.74
-0.27242818	78617	6.26	1255429	100.00

Discretization 1 vs. Discretization 2

In order to compare the results from the two approaches, we will use the variable “Tsbal”, which gives us the total open balances. Table 7 shows the initial ten groups of “Tsbal” made by using proc rank in discretization 2. Table 8 shows the significant groupings retained in discretization 2 from the original 10 groupings. And table 9 shows the user-defined groups for “Tsbal” from discretization 1. It is evident that the user-defined process was not good at differentiating the significant groups from the insignificant, and thus has many more groupings than needed, which would have a bad effect on the final model.

Table 7: TSBAL Initial Ranks

Rank for Variable TSBAL				
rank	Frequency	Percent	Cumulative Frequency	Cumulative Percent
0	125503	10.00	125503	10.00
1	125565	10.00	251068	20.00
2	125523	10.00	376591	30.00
3	125553	10.00	502144	40.00
4	124210	9.89	626354	49.89
5	126888	10.11	753242	60.00
6	125556	10.00	878798	70.00
7	125540	10.00	1004338	80.00
8	125544	10.00	1129882	90.00
9	125547	10.00	1255429	100.00

Table 8: TSBAL Significant Ranks

Rank for Variable TSBAL				
rank	Frequency	Percent	Cumulative Frequency	Cumulative Percent
0	125503	10.00	125503	10.00
1	251088	20.00	376591	30.00
3	125553	10.00	502144	40.00
4	124210	9.89	626354	49.89
5	503528	40.11	1129882	90.00
9	125547	10.00	1255429	100.00

Table 9: Discretization 1 of TSBAL

Analysis Variable : goodbad				
ordTSBAL	N Obs	N	Mean	Std Dev
1	358209	358209	0.1499320	0.3570053
2	202593	202593	0.1807910	0.3848459
3	163496	163496	0.1897233	0.3920833
4	124338	124338	0.1871431	0.3900279
5	99498	99498	0.1856922	0.3888600
6	76739	76739	0.1875448	0.3903508
7	58424	58424	0.1854204	0.3886415
8	44063	44063	0.1856433	0.3888229
9	32957	32957	0.1868192	0.3897722
10	24844	24844	0.1877314	0.3905054
11	33242	33242	0.1886469	0.3912337
12	10510	10510	0.1869648	0.3899018
13	7999	7999	0.1885236	0.3911541
14	6077	6077	0.1968076	0.3976184
15	4650	4650	0.1864516	0.3895126
16	3495	3495	0.1888412	0.3914384
17	2751	2751	0.1864776	0.3895624
18	1544	1544	0.1917098	0.3937735

Figure's 10 and 11 compare the results: the user-defined relationship does not seem to be as strong as the one from discretization 2. The discretization 2 plot bounces up and down which neutralizes any significant effects of one predictor on the other. Although both plots don't seem to be quite linear, discretization 2 plot has more hope.

Figure 10: Tsbal (Discretization 1) vs. Mean Goodbad

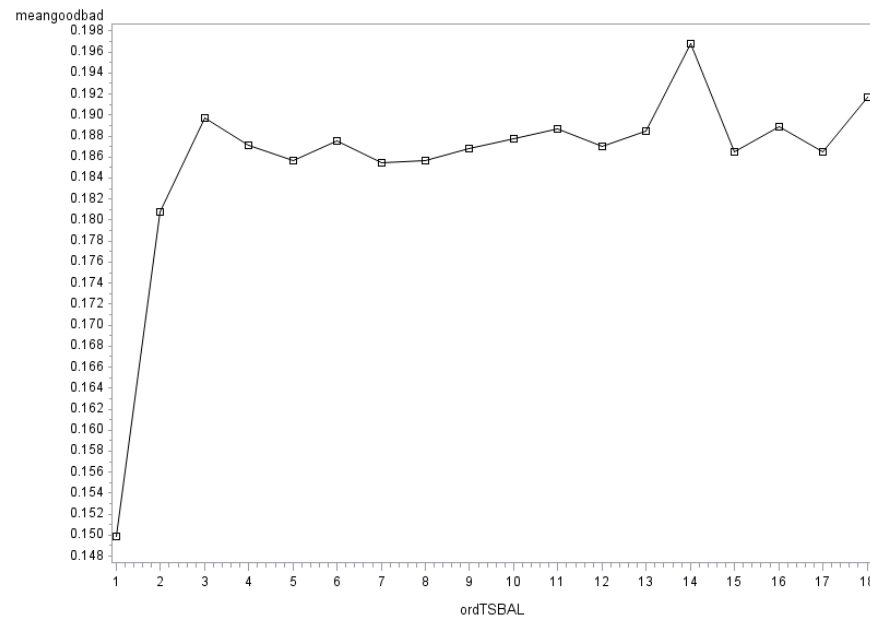


Figure 11: Tsbal (Discretization 2) vs. Mean Goodbad

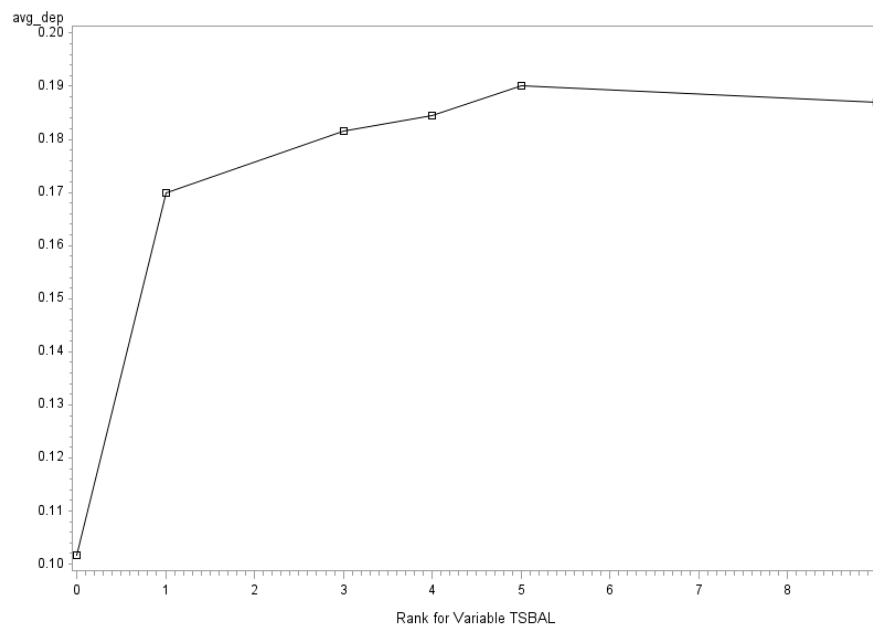


Table 10 shows the differences in the mean, and range of the two methods. We can see how the difference in the mean values is not much different, but the ranges and standard deviations differ a lot. Discretization 1 was a more intuitive process, whereas discretization 2 was more

mechanical. Hence, discretization 1 took longer than discretization 2; in the real world more mechanical procedures would be more cost-effective as long as the quality isn't sacrificed. So far, we don't see significantly better results one way or the other. Overall, some transformations were captured better with the discretization 1 method, whereas the others were better using the discretization 2 method. After the final model we'll be able to better judge which method provided us with stronger relationships.

Table 10: Discretization 1 vs. Discretization 2

Variable	N	Mean	Std Dev	Minimum	Maximum
ordeqtsbal	1255429	3.8011708	1.6004035	1.0000000	6.0000000
odseqtsbal	1255429	0.2139315	0.0353454	0.1131876	0.2346073
lodseqtsbal	1255429	-1.5610992	0.2119024	-2.1787084	-1.4498422
ordTSBAL	1255429	3.9004675	3.2094333	1.0000000	18.0000000
oddstsbal	1255429	0.2136545	0.0239098	0.1763765	0.2450318
loddstsbal	1255429	-1.5501402	0.1182750	-1.7351343	-1.4063675

By the end of these two processes we had 405 variables.

Modeling

By this point we had a cleaned out data set with several transformations of each variable, and we were ready to begin the variable selection process. But first we split the data set into three parts; 60% was retained as the training data-set, and 20% each was allocated for test data-sets, so we could validate our model later based on a different set of observations to ensure it would work universally. This is an important step because the whole purpose of predictive modeling is to ensure the model can be used for all applicable datasets, and not just the one that was used to create the model.

We used a seed (123456) to ensure the split could be replicated if needed, and performed the split using simple random sampling without replacement. We will determine whether our model and its results are duplicable or not based upon the results from the test data-sets. If the test datasets return similar results as the training data-set we will know that our model is universally applicable. Whereas an extreme difference in results would suggest we have an over-fitted model that would not work well in the real world with new and changing data.

Variable Selection

We utilized the built-in stepwise selection method in SAS to help narrow down our variable pool from 405 to about 20 or less. For the modeling process we only used the Train Dataset. It would've been quite a time consuming task to find the best variables manually digging through 405 variables, hence we decided to use a stepwise selection method which only keeps variables at the user's specified significance level. Still fitting a stepwise selection model to 405 variables is quite a computationally heavy task, so we broke it down into pieces and helped SAS along the way by fitting a few models each time, and collecting the best variables along the way.

The stepwise selection adds a new variable one at a time, and with the entry of each variable again tests if all the variables in the model are still significant or not – unlike the forward selection. Only if the variable had a low enough p-value was it retained in the model. Given so many variables to choose from, and the need for high precision, we used the “sle” (significance level (for) entry) and “sls” (significance level (to) stay) option and specified them to both be 0.0001. This helped quickly narrow down our huge variable pool.

At times we came across variables with multiple significant transformations. We couldn't allow redundant versions in our model, so we utilized other methods to decide which variable to finally keep.

Which Transformation to Keep: From the individual variable transformations we could only keep one in our final model. Where necessary three approaches were used to determine the best transformation:

1. We took into account the Chi-Square for each transformation, and the best Chi-Square transformation was retained.
2. If the Chi-Squares were comparable, and we only had two transformations remaining to select from, we kept the form that would make most sense in context.
3. If we had more than two variables with comparable Chi-Squares then we plotted each transformation against the response variable and chose the transformation with the most linearized graph.

By the end of our variable selection process we were left with the following 19 variables (originals displayed):

Table 11: Original Variables Kept in Final Model

Variable	Variable Description
BNKINQ2	= '# OF BANK PROM INQS /IN PAST 6 MONTHS';
BRAVGMOS	= 'AVG # MONTHS/BANK REV ACCT OPEN';
BRBAL75	= '# OF BANK REV ACCTS/BAL > 75% HI CREDIT';
BRCRATE1	= '# OF BANK REV ACCTS/CURRENTLY SAT';
BRMAXB	= 'MAX OPEN BANK REV BAL';
BROPEN	= '# OF "OPEN"/BANK REV ACCTS';
BRWCRATE	= 'BANK REV /WORST CURRENT RATING';
CRATE2	= '# OF ACCOUNTS/CURRENTLY 30 DAYS';
CRDPATH	= 'AGE OF OLDEST TRADE';
LOCINQS	= '# OF LOCAL INQ/IN PAST 6 MO';
OT24PTOT	= '% OF OPN TRADES IN 24/ TO TOT OPN TRADES'
PDBAL	= '# OF ACCTS WITH/PAST DUE BALANCES';
PRAGE	= 'AGE OF MOST RECENT/PUBLIC RECORD';
TADB	= 'TOTAL/AVERAGE DEBT BURDEN';
TADB25	= '# OF OPEN TRADES WITH BAL < 25% HIGH C';
TPOPEN	= '% OF ALL ACCTS/ THAT ARE "OPEN"';
TR7924	= '# OF ACCOUNTS/BAD DEBT IN PAST 24 MO';
TRATE2	= '# OF ACCOUNTS/30 DAYS LATE';

Logistic Regression

Our response variable is a binary variable with a '0' indicating a credit-worthy customer and a '1' indicating bad-risk. Models with a binary response variable cannot have normalized errors, or constant variances. Since the response can only take values of 'zero' or 'one', the response function has constraint on it. Hence linear regression cannot be fitted to such a scenario. Furthermore, a binary response variable's response function is generally nonlinear, and follows the S-shaped Logistic Response Function.

So instead of fitting a linear model which looks like:

$$Y = \beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k$$

with possible outcomes ranging from $-\infty$ to ∞ , we fit:

$$\text{Log}(\pi) = \beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k$$

where the response is limited to range between 0 and 1.

The output below shows the overall test of significance of regression (Testing Global Null Hypothesis). For logistic models instead of having a F-Statistic, we have Likelihood Ratio, Score and the Wald Statistic – all of which are significant at a p-value of <.0001.

Table 12: Significance of Model

Testing Global Null Hypothesis: BETA=0			
Test	Chi-Square	DF	Pr > ChiSq
Likelihood Ratio	185884.133	19	<.0001
Score	192873.497	19	<.0001
Wald	118931.818	19	<.0001

Analysis of Maximum Likelihood Estimates					
Parameter	DF	Estimate	Standard Error	Wald Chi-Square	Pr > ChiSq
Intercept	1	0.9973	0.0666	224.1422	<.0001
loddspdbal	1	0.4690	0.00518	8199.8873	<.0001
TADB	1	0.8308	0.0171	2349.8872	<.0001
loddbrwcrate	1	0.4357	0.00527	6842.9250	<.0001
oddsbravgmos	1	3.5589	0.0657	2936.8058	<.0001

The Analysis of Maximum Likelihood Estimates displays the individual variables with their estimates, standard errors, Wald Chi-Squares, and their associating p-values. All the variables retained in our model were significant with a p-value of <.0001.

Since the logistic regression procedure in SAS already calculates the log od odds for each variable plugged in, the estimates we are looking at are actually all log of odds of whatever variable is plugged in. The additional transformations we took were just to find the best mathematical relationship between the individual variable and the response. So if we take the variables “TADB” which is short for total/average debt burden, the interpretation of it’s estimate in context would be that for every extra unit of “TADB” the customer’s chances of default dropped by 0.8308. To solve for the actual probability we still needed to solve the function

$$\pi = \frac{e^{\beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k}}{1 + e^{\beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k}}$$

Table 13: Odds Ratio Estimates

Odds Ratio Estimates and Wald Confidence Intervals				
Effect	Unit	Estimate	95% Confidence Limits	
loddspdbal	1.0000	1.598	1.582	1.615
TADB	1.0000	2.295	2.219	2.374
loddnbrwcrate	1.0000	1.546	1.530	1.562
oddsbravgmos	1.0000	35.126	30.884	39.951

The above table displays a sample of the Odds Ratios for our selected variables. For every one unit of change in the variable “TADB” the odds of default increased by 2.295.

The table below shows how our training dataset overall did in terms of “goodbad”. We see 82% of our training data represented the credit-worthy customers (goodbad = 1). This follows our initial analysis of our complete data set where 85% of our customers were classified as good-risk. So our training data-set is a good representation of the complete data-set.

Table 14: Distribution of Good and Bad

Response Profile		
Ordered Value	goodbad	Total Frequency
1	0	621090
2	1	132157

Model Summary

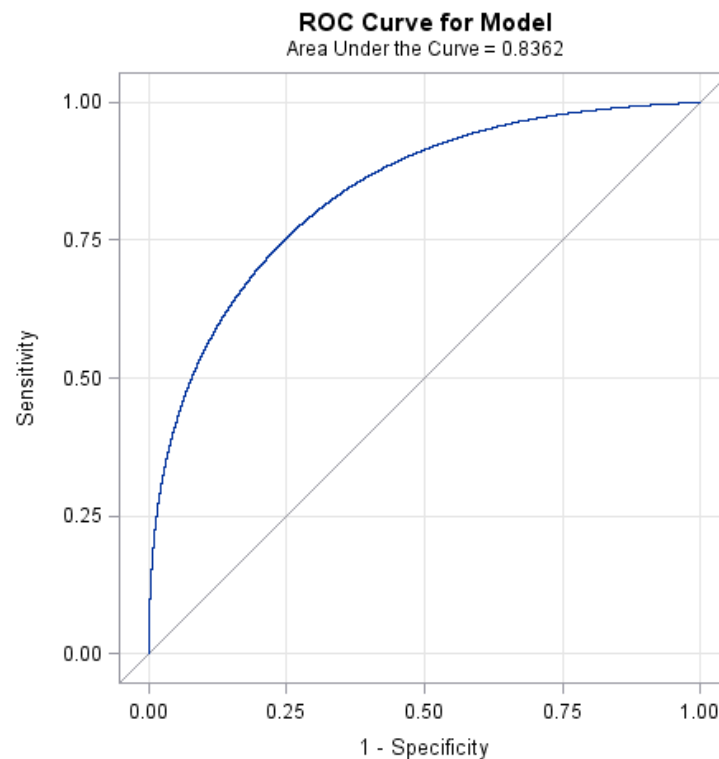
Table 15: Predicted vs. Observed

Association of Predicted Probabilities and Observed Responses			
Percent Concordant	83.6	Somers' D	0.672
Percent Discordant	16.4	Gamma	0.672
Percent Tied	0.0	Tau-a	0.195
Pairs	82081391130	c	0.836

To determine our model's performance for the dataset at hand, our first choice was to evaluate our model's associated c-statistic, which is based upon concordant and discordant pairs. SAS automatically makes all possible pairs among observations and then compares the predicted values for the pairs against the observed. If both the pairs were predicted correctly then we have a concordant pair, otherwise it's discordant. Hence a c-statistic of 0.50 would mean our model is as good as a coin flip, whereas a c-statistic of higher than (or even below) 0.50 means we have

captured some significant relationship between the response and the regressor variables. And as it approaches 1 we reach the maximum predictive power of a model. So in our case, a c-statistic of 0.836 is a good indication. We can conclude that our model has a good predictive distinction between the good (0) and the bad(1) customers. This c-statistic corresponds to the ROC curve (Receiver Operating Characteristics). Observe that the c-statistic is the area beneath the ROC curve. This helps as an aid to visualize the strength of our predictive model. The 45 degree line represents the hypothetical 0.50 c-statistic against which models are compared. The ROC curve uses all the correct predictions (classifications) and incorrect predictions to compute the curve. Hence it is not a good aid in evaluating a universal model where we may not have the correct representations of a real-world model.

Figure 12: ROC Curve



Model Evaluation

We evaluated our model using four evaluation methods. Each has its advantages and disadvantages. First we defined our matrix which would determine how our model performed overall and will finally aid in calculating our profit (for which only false-negative and sensitivity will be used). For every correct prediction of good risk, we will earn our client \$250, and for every bad-risk predicted incorrectly as good-risk our client will lose half of that customers credit line. But for the best performance of our model we need to determine the optimal cut-off point (discussed later).

Table 16: Categorization of Predictions vs Observed

		Predicted	
		Positive (1)	Negative (0)
Observed	Positive (1)	Specificity (True BAD risk predicted correctly)	False Negative (Lose 0.5 credit line – Type II Error: Classified as good risk but truly a bad risk)
	Negative (0)	False Positive (Type I Error – Classified as bad, but truly a good risk = opportunity cost)	Sensitivity (\$250 – true GOOD risk classified correctly)

Hosmer and Lemeshow Test

Table 17: Hosmer and Lemeshow

Partition for the Hosmer and Lemeshow Test					
Group	Total	goodbad = 1		goodbad = 0	
		Observed	Expected	Observed	Expected
1	75325	928	991.04	74397	74333.96
2	75328	1669	1886.37	73659	73441.63
3	75327	2835	2983.69	72492	72343.31
4	75323	4384	4448.37	70939	70874.63
5	75325	6341	6364.00	68984	68961.00
6	75324	9105	8821.62	66219	66502.38
7	75326	12521	12078.93	62805	63247.07
8	75325	17270	16996.40	58055	58328.60
9	75325	26370	26509.72	48955	48815.28
10	75319	50734	51079.02	24585	24239.98

Hosmer and Lemeshow Goodness-of-Fit Test		
Chi-Square	DF	Pr > ChiSq
82.1958	8	<.0001

The Hosmer-Lemeshow test classifies observations in groups (by default 10) by estimated probabilities of success. It does this by comparing observed and predicted frequencies. The test forces observations into deciles of risk and returns a lack-of-fit statistic, in our case 82.20 with a p-value of <.0001, which means that our model is not an adequate fit to the data. But since the Hosmer-Lemeshow test isn't good at testing models with innately binary or ordinal variables we ignore it in this instance.

Classification Table

Table 18: Classification Table

Classification Table									
Prob Level	Correct		Incorrect		Percentages				
	Event	Non-Event	Event	Non-Event	Correct	Sensitivity	Specificity	False POS	False NEG
0.000	132E3	0	621E3	0	17.5	100.0	0.0	82.5	.
0.100	116E3	362E3	26E4	16273	63.4	87.7	58.2	69.1	4.3
0.200	91564	501E3	12E4	40593	78.7	69.3	80.7	56.7	7.5
0.300	72473	559E3	61941	59684	83.9	54.8	90.0	46.1	9.6
0.400	58039	587E3	34555	74118	85.6	43.9	94.4	37.3	11.2
0.500	46235	602E3	19588	85922	86.0	35.0	96.8	29.8	12.5
0.600	36001	61E4	10612	96156	85.8	27.2	98.3	22.8	13.6
0.700	26234	616E3	5341	106E3	85.2	19.9	99.1	16.9	14.7
0.800	16696	619E3	2112	115E3	84.4	12.6	99.7	11.2	15.7
0.900	7384	621E3	487	125E3	83.4	5.6	99.9	6.2	16.7
1.000	0	621E3	0	132E3	82.5	0.0	100.0	.	17.5

The classification table is a great aid in determining the optimal cut-off point (Prob Level in the above table). The column correct shows the percentage of overall correctly classified events. So for a cut-off point of 0.3 we have 83.90% correctly classified events, and 9.6% false negatives (bad-risk classified as good-risk, which results in half the credit limit gone).

Sensitivity (Earn \$250 – true GOOD risk classified correctly) for our cut-off of 0.3 is found by:

$$\text{Sensitivity} = \frac{\text{True Positive}}{\text{Total Actual Positives}} = \frac{\text{True 1}}{\text{Total Actual 1's}} = \frac{72473}{72473 + 61941} = 0.54$$

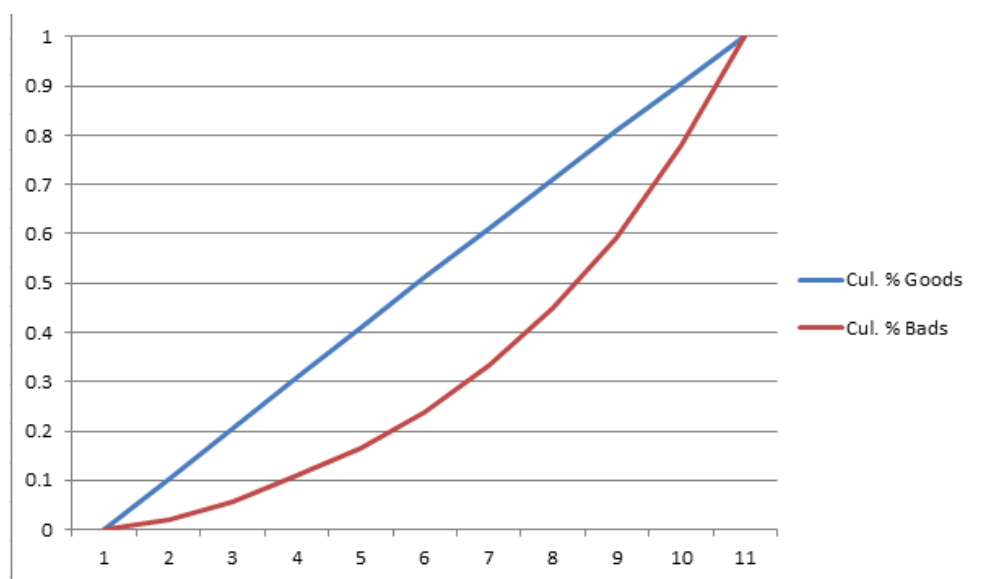
Similarly, specificity (True bad-risk predicted correctly) for our cut-off of 0.3 is found by:

$$\text{Specificity} = \frac{\text{True Negative}}{\text{Total Actual Negatives}} = \frac{\text{True 0}}{\text{Total Actual 0's}} = \frac{559000}{559000 + 59684} = 0.90$$

So the classification table gives us helpful insights at each probability of default level (cut-off point). It is a good method to evaluate Global classification rates and assess the overall predictability of our model. But since we cannot justify the classification table's inherent assumption that Type I and Type II error have the same cost, we have to continue using other evaluation methods.

Kormogorov-Smirnov Statistic

Figure 13: K-S Plot



The K-S graph is another way to visualize the strength of the model. It shows the maximum

Table 19: K-S Table

	Cul. % Goods	Cul. % Bads
0	0.00%	0.00%
1	10.36%	2.03%
2	20.66%	5.49%
3	30.87%	10.79%
4	41.06%	16.61%
5	51.19%	23.80%
6	61.21%	33.27%
7	71.14%	44.85%
8	80.94%	59.27%
9	90.54%	78.06%
	100.00%	100.00%

distance between the cumulative percentages of successes against failures. The table on the left shows the different deciles, and how the graph was plotted. So for decile #3 the percentage of true good-risk predicted is 30.87%. In general we notice that the cumulative percentage of bad-risk predicted lags behind, but starts catching up around decile #7.

When using the K-S test for determining the optimal cut-off point we look for the biggest “gap” or difference between the two curves. Which in our case is around decile 7.

This test doesn’t provide any sort of indication as to what our final profit may be, so we need to continue on to the next evaluation method.

Profitability of the Model

Table 20: Profitability (Valid 1)

outcometype	pct	n	profit	pper1000
ERROR1	0.0621866	15650	\$-15,535,338.00	\$-992,673.35
ERROR2	0.1284977	32338	\$0.00	\$0.00
VALID1	0.1132352	28497	\$0.00	\$0.00
VALID2	0.6960805	175177	\$43,794,250.00	\$250,000.00
	1	251662	\$28,258,912.00	\$112,289.15

As mentioned earlier, for our final profitability we will only use false negatives (Error 1) and sensitivity (Valid 2). The above table shows our final profit based on the first validation file. The “ERROR2” is the true good predicted as bad, and the “VALID1” is true bad predicted as bad. Again, in the real world these two may need to be taken into consideration as well, but since it is hard to define per unit cost or gain to them we left them out for this project.

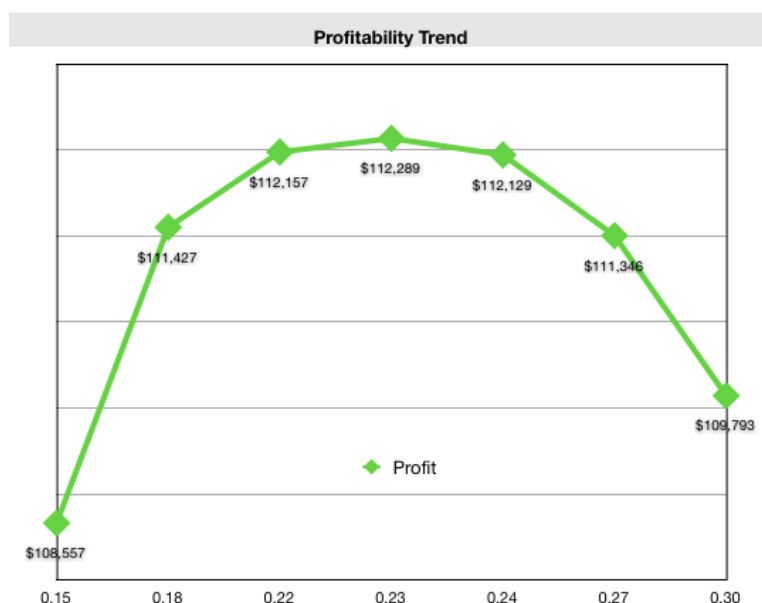
Based on the table above we would gain our client \$43,794,250 based on our true good-risk predicted correctly. On the other hand we will lose our client \$15,535,338 due to predicting actual bad-risk customers as good-risk. Overall, the company would earn \$28M, which is roughly \$112K profit per 1000 customers.

We used the validation 2 file as well to make sure our results were universally applicable to different data-sets and observations (the results are displayed below). We don’t observe a major drop (or gain) in profit per 1000 even though this is an entirely different subset of the original data-set. So we can conclude that our model works well for different data-sets, and thus is universal.

Table 21: Profitability (Valid 2)

outcometype	pct	n	profit	pper1000
ERROR1	0.0629686	15773	\$-15,452,979.00	\$-979,710.83
ERROR2	0.1294343	32422	\$0.00	\$0.00
VALID1	0.113753	28494	\$0.00	\$0.00
VALID2	0.6938441	173801	\$43,450,250.00	\$250,000.00
	1	250490	\$27,997,271.00	\$111,770.01

Figure 14: Profitability Trend



The graph above shows the final method we used in determining our cut-off point. We evaluated different cut-off points for our model and plotted them against the returns. Since the highest returns are at a cut-off of 0.23 that is our optimal cut-off point.

Transaction Performance

The table below shows a contingency table for a few SIC codes by Good(0) or Bad(1) risk. Right away we notice that the Good column always has more transactions, but that's only because we have more good-risk observations than bad risk. If we see a huge difference between the two groups, then either there is a significant relationship between the SIC code and credit-risk, or we are just lacking meaningful observations for that certain SIC code. For example, customers who spend money on government services (paying tag-titles etc) tend to be classified as good-risk 83.42% of the time.

Table 22: Contingency Table for Transactions

Frequency			
Percent			
Row Pct			
Col Pct			
Table of TRANSIC by goodbad			
TRANSIC(transic)	goodbad		
	0	1	Total
9222 - FINES	23	6	29
	0.01	0.00	0.01
	79.31	20.69	
	0.01	0.01	
9311 - TAX PAYMENTS	90	19	109
	0.04	0.01	0.04
	82.57	17.43	
	0.04	0.04	
9399 - GOVERNMENT SERVICES	1142	227	1369
	0.45	0.09	0.54
	83.42	16.58	
	0.55	0.51	

Table 23: T-Test for Pet Foods and Supplies

goodbad	N	Mean	Std Dev	Std Err	Minimum	Maximum
0	708	58.3079	82.9819	3.1186	0.0100	1369.3
1	158	66.9832	123.1	9.7948	4.2300	1219.0
Diff (1-2)		-8.6752	91.5924	8.0588		

goodbad	Method	Mean	95% CL Mean		Std Dev	95% CL Std Dev	
0		58.3079	52.1850	64.4309	82.9819	78.8732	87.5455
1		66.9832	47.6366	86.3297	123.1	110.9	138.4
Diff (1-2)	Pooled	-8.6752	-24.4924	7.1420	91.5924	87.4703	96.1252
Diff (1-2)	Satterthwaite	-8.6752	-28.9514	11.6010			

Method	Variances	DF	t Value	Pr > t
Pooled	Equal	864	-1.08	0.2820
Satterthwaite	Unequal	190.01	-0.84	0.3998

Equality of Variances				
Method	Num DF	Den DF	F Value	Pr > F
Folded F	157	707	2.20	<.0001

The table above shows t-test output for SIC code 5995 (PET SHOPS, PET FOODS). Since our variances p-value is less than our alpha level, we will interpret the Satterthwaite differences. Based on our Satterthwaite differences p-value of 0.3998 there is no significant difference between good-risk and bad-risk people who spend on pet foods and supplies.

Limitations and Weaknesses

The modeling process itself was not the most computationally and code intensive, but the actual variable preparation took the longest. Hopefully as data capturing improves we will not have to deal with missing values, and used imputation methods. Imputation methods are a useful tool in “rescuing” data, but at the same time one cannot truly take into account what effect they might’ve had on the data-set.

In actual business environments the time allowed to come up with such models is a lot shorter, and requires more precision at the same time. So a more Macro-tized approach to the computationally intensive parts of the project, such as the discretization process, may help fasten results.

Also, not taking the lost opportunity-cost and specificity is a limitation of the data and information provided to us, but seems like it would be a crucial factor in a real business.

Cost of Simplicity

In the real-world when credit-lending companies are deciding whether to issue a person credit or not, they need to be able to successfully make that decision instantly. If not, they may lost the customer to another company. Thus predictive models, such as ours, need to be kept as parsimonious as possible, so that the information required from the customer to use the model is not exhaustive.

The other kind of cost a company may need to take into account is the actual time taken to build the model. As mentioned earlier the longest part of the project was to perform the two separate discretization processes. Whereas in real life only one would be needed. If the dataset wasn't as big as ours, and maybe even more than one person was assigned to the project, then discretization 1 would be the ideal choice. It is more time-consuming since it is more user defined, but based upon our final model it yielded better variables then discretization 2. 13 out of our 19 final variables came from the discretization 1 process, whereas only 3 came from the discretization 2 process. The remaining three were retained in their original form. So we can see that a discretization process gives better results and hence even though time-consuming is needed for good modeling.

Conclusions

So our final model is:

Log of Odds(GOODBAD (event='1')) = $0.9973 + 0.4690 \cdot \text{loddspdbal} + 0.8308 \cdot \text{tadb} + 0.4357 \cdot \text{loddsbrwcrate} + 3.5589 \cdot \text{oddsbravgmos} + 0.4294 \cdot \text{loddsbrcrate1} + 4.7817 \cdot \text{oddsbrbal75} + 0.4700 \cdot \text{loddstpctsat} + 0.6013 \cdot \text{lodseqbrmaxb} - 0.0998 \cdot \text{tr7924} + 0.6512 \cdot \text{loddsbropen} + 0.4185 \cdot \text{loddsot24ptot} + 1.9397 \cdot \text{oddstrate2} + 2.8641 \cdot \text{oddslocinqs} - 0.0383 \cdot \text{tadb25} + 1.0654 \cdot \text{odseqprage} - 1.9796 \cdot \text{odseqtpopen} + 1.4520 \cdot \text{oddsbnkinq2} + 0.5034 \cdot \text{oddscrate2} + 0.4813 \cdot \text{loddscrdpth}$

Analysis of Maximum Likelihood Estimates					
Parameter	D F	Estimate	Standard Error	Wald Chi-Square	Pr > Chi Sq
Intercept	1	0.9973	0.0666	224.1422	<.0001
loddspdbal	1	0.4690	0.00518	8199.8873	<.0001
TADB	1	0.8308	0.0171	2349.8872	<.0001
loddsbrwcrate	1	0.4357	0.00527	6842.9250	<.0001
oddsbravgmos	1	3.5589	0.0657	2936.8058	<.0001
loddsbrcrate1	1	0.4294	0.00849	2560.0638	<.0001
oddsbrbal75	1	4.7817	0.0426	12603.8192	<.0001
loddstpctsat	1	0.4700	0.00840	3133.4582	<.0001
lodseqbrmaxb	1	0.6013	0.00993	3668.6562	<.0001
TR7924	1	-0.0998	0.00212	2213.0370	<.0001
loddsbropen	1	0.6512	0.0147	1964.8702	<.0001
loddsot24ptot	1	0.4185	0.0107	1539.7943	<.0001
oddstrate2	1	1.9397	0.0539	1293.7667	<.0001
oddslocinqs	1	2.8641	0.0759	1424.0579	<.0001
TADB25	1	-0.0383	0.00168	520.2673	<.0001

odseqprage	1	1.0654	0.0447	567.0257	<.0001
odseqtpopen	1	-1.9796	0.0812	593.7518	<.0001
oddsbnking2	1	1.4520	0.0530	749.8528	<.0001
oddscrate2	1	0.5034	0.0191	695.9989	<.0001
loddscrdpth	1	0.4813	0.0262	337.4793	<.0001

We only notice three variables decreasing the odds of default: TR7924, TADB25 and odseqTPOPEN. All other parameters increase the log of odds of the customer defaulting. All our parameters are significant with p-values <.0001. This is our final model.